LSTAR PROTOCOL

BY

DOUGLAS QUIGG

THE QUERY PROTOCOL STEP BY STEP
EVERY CLAIM, WHAT STANDS BEHIND IT, AND WHAT IT WANTS

2026

© 2026 Douglas Quigg
AGPL-3.0-or-later OR LicenseRef-Commercial OR LicenseRef-Educational

CONTENTS

1 THE LSTAR PROTOCOL 1
1.1 Entries 1

2 THE QUERY PROTOCOL TABLE 3
2.1 The rule the protocol is held to 3
2.2 The inputs 3
2.3 The files 4
2.4 The table 7
2.5 The algebra of the protocol 21
2.5.1 P1. The ask and its bound (Q1, Q2, Q3) 22
2.5.2 P2. The gate (Q4) 22
2.5.3 P3. The order of asks (Q5) 23
2.5.4 P4. Contention (Q7) 23
2.5.5 P5. One ask, many answers (Q15) 23
2.5.6 P6. The rank, read against its own spread (Q16, Q17) 24
2.5.7 P7. A form asked of the part (Q16) 24
2.5.8 P8. Held against NVIDIA's compiler (Q17) 25
2.5.9 P9. The part's own bound (Q1, Q16, Q17) 25
2.5.10 P10. Absurdity from a part of a set (Q19) 28
2.5.11 P11. The ruleset, derived ask by ask (Q4, Q11, Q18, Q19) 29
2.5.12 P12. Categories, and the coherence set C* (P10, P11) 33
2.5.13 P13. The sets, open-ended, and what they collapse (P2, P5, P10, P11, P12) 36
2.5.14 P14. A system learned, and the writers that read it (P8, P9, P11, Step 9) 49
2.6 The protocol 50
2.7 Open 52
2.8 The run, a database (Q20) 58
2.9 The query, a crate 62
2.10 The frame asked (Open 6.3) 66
2.11 The fixed registers asked (Open 6.2) 67
2.12 The frame forms asked (Open 6.4, the frame) 68

3 ANGLUIN'S L* AND THE LSTAR PROTOCOL 71
3.1 L* 71
3.2 The terms, side by side 71

3.3 Where the protocol asks past L* 72
3.3.1 A living language has no last state 72
3.3.2 The Teacher is not minimally adequate 73
3.3.3 The sets in place of the conjecture 74
3.3.4 What is bounded when nothing ends 74
3.4 Status 75

BIBLIOGRAPHY 77

1

THE LSTAR PROTOCOL

Purpose: The query protocol step by step: what each step asks, the algebra it holds to, what it does, what it wants, what was tried and what it gave, its status, and the next move.

Scope: `workbooks/Lstar_protocol/`. Statuses follow the engine workbook's README.md, and the engine the protocol asks for is `workbooks/engine/`.

1.1 ENTRIES

file	what it holds
query_protocol_table.md	the query protocol step by step: what each step asks, the algebra it holds to, what it does and wants, what was tried and what it gave, and the run behind every status
angluin.md	Angluin's L* set beside the protocol: the minimally adequate Teacher, the observation table, termination and its polynomial bound, each term's place in the protocol, and where the protocol asks past it, a living language among them

2

THE QUERY PROTOCOL TABLE

Purpose: The query protocol step by step: what each step asks, the algebra it holds to, what it does, what it wants, what was tried and what it gave, its status, and the next move.

Scope: every step between a question put to a target and a branch consuming the answer: the ask, the unbound and bound passes, the gate, the order of asks, the contention read, cross-branch comparison and the record. Its inputs are the entries of `Lstar.klq`, the bridge between languages, and of each language's `.klm`, and every step gives back entries of them. The protocol's form and its mnemonics are in gnascor.md, and its open work is in this table: each step's next move, the protocol and its open work after the algebra, and the run as a database after them. Statuses follow README.md, and every status names the run behind it.

2.1 THE RULE THE PROTOCOL IS HELD TO

Gate, then rank. Never one score. A relation holds or it does not, and that answer carries no noise. A cost is measured and every cost carries noise. The gate decides which candidates are admissible and the rank orders whatever survives, and the two are never added together. Kept apart, a noisy cost can cost speed and can never cost correctness. The engine already holds this rule in its own descent: survival is a conjunction, order cannot change a conjunction, and a planner that steers badly costs speed and never a wrong survivor (`src/cu/engine/nbody/orior/orior_descent.h`).

Steer on what is known to be true. Every relation the ladder holds is arithmetic every system that computes agrees about, and the host computes it. The gate is planned against that ground truth on the host, and a target is asked only the cases the plan placed.

2.2 THE INPUTS

The inputs are entries. `Lstar.klq` is keyed by the schema's form names, which are gnascor's and no target's. For each key it holds the pairs of forms whose sameness the text leaves open, the cases put to them, and the verdict: closed as two operations with the case that witnessed it, or open with its count of cases alike. A language's `.klm`, `cu.klm` or `sass.klm`, keys that language's names to the bridge, each entry naming its relation on its own line. An address is a key of the bridge read through a language's map, and a map from one language to another is read through the bridge, `cu.klm` to `Lstar.klq` to `sass.klm`. `klq_write` (`src/cu/transpiler/lstar/protocol`, run by `orior protocol write bridge`) writes `Lstar.klq` and each given language's `.klm` beside

the rulesets, the forms of each line in `src/cu/types/file_defs/klq` and `klm`, and no step reads one.

A reading of a part of a set is 1, 0 or gray. No set of entries is known to be whole. A witnessed relation that holds of a part holds of every set holding that part, and its 1 is a verdict; where it does not hold, a file added can make it hold, and the reading is gray. A relation of absence reads the other way: a name given is given in every set holding it, and that 0 is a verdict, and a name not given is gray. gray is gnascor's state: no answer, and not void, which is an answer of no (P1, P10).

2.3 THE FILES

Every file the protocol is made of, by what it holds. Paths are from the root of the tree.

The ask and the bus. `src/cu/transpiler/lstar/protocol/query/`

file	holds
`host_entry.h`	the two relations a host is reached by, a word put and a word read at an address, and the kind an address shows and the width a sizing register names
`query_ask.h`, `query_ask.c`	one ask: an address, a qualifier, the cost read off a clock found by asking, and the bit with its kind (Q1, P1)
`query_interface.h`, `query_interface.c`	asks put from inside the interface in probes it can lose, an ending kept as the answer of the address that caused it (Q1)
`query_walk.c`	the program a walk of asks runs as, inside the cell
`classify_walk.c`	the program a classify walk runs as: the kind ask at each address of a span
`bus_enum.h`, `bus_enum.c`	the enumeration region found in a run of kinds: a fixed identifier beside a live sizing register at one stride
`bus_enum_walk.h`, `bus_enum_walk.c`	the classify walk driven over a span in probes, and the region read from it
`answer_read.h`, `answer_read.c`	what the part answered of an ask, alike, apart, refused or nothing against the host's bracket at the result's width, and gray where no answer decides it (P11)

The channel. `src/cu/transpiler/lstar/protocol/teacher/`

file	holds
`run_channel.h`, `run_channel.c`	the run channel: a container in, eight words put, four read back, and an outcome of answered, nothing, illegal or no channel (Q1, Q15)

The gate. `src/cu/transpiler/lstar/protocol/gate/`

file	holds
`chain_build.h`, `chain_build.c`	every arrangement of primitives that answers a relation's cases (Q4)

file	holds
`survivors.h`, `survivors.c`	the candidates that fail no case, the descent's order with the most-failed case first, and the price of an order (P2, Q4, Q11)

The ladder. `src/cu/transpiler/lstar/protocol/counterexample/`

file	holds
`ladder.h`	the relations known before anything is met, written as cases

The order and the rank. `src/cu/transpiler/lstar/protocol/order/`

file	holds
`ask_order.h`, `ask_order.c`	the known order of asks, its solve, and the read of whether the links contend (Q5, Q7, P3, P4)
`query_order.h`, `query_order.c`	the known order put through the protocol, each cost an exact rational (Q5)
`stem_group.h`, `stem_group.c`	members grouped on anchors by a rule the members alone fix (Q13)
`scheduler.h`, `scheduler.c`	the gray entries of every set as a queue, the ask that settles the most put first, carried in rounds through the channel (P13)

The record. `src/cu/types/file_defs/kqr/`

file	holds
`kqr.h`, `kqr.c`	R: every ask with its answer, the part and the size it was read on, read whole when opened and written whole when closed, an ask it holds not put again (Q9, Q10, P13)

The interface. `src/cu/transpiler/lstar/interface/`

file	holds
`interface.h`, `interface.c`	the probe runner: a probe asks the target one question in a process the interface can lose, and its ending is read, exited, signaled, faulted or out of time, the fault named the same on every host
`interface_names.c`	the name of each ending and each fault

The machine files, written. `src/cu/transpiler/lstar/parser/`

file	holds
`khw_write.c`	a part's forms learned by asking the part and its `.khw` written from the answers: the slot found, the relation asked, the fields turned and the runs set to the two registers (P8, P9, Q16, Q17)

file	holds
`kdm_write.c`	a part's `.kdm`, the arrangements of primitives that produce each operator, found from the relations alone, costs carried forward across a rewrite (Q2)

The part. `src/cu/transpiler/vendor_bin_layouts/nvidia/`

file	holds
`monolith.cu`, `monolith.h`	one program holding every base precept between tags, its costed chains and their layout (Q2, Q17)

The relations of the text. `src/cu/types/file_defs/readers/`

file	holds
`ruleset_core_relation.h`	the absurd relations among the entries of any files read together, and the map from one set to another (Q19, P10)
`ruleset_flat.h`, `ruleset_flat.cu`	the files a ruleset is read from, read into the caller's memory, and what each relation says

The bridge. `src/cu/transpiler/lstar/protocol/table/`

file	holds
`klq_write.cu`	the bridge written: `Lstar.klq` and each language's `.klm`, run by `orior protocol write bridge`
`klq_identity.cu`	a candidate chain sifted through the identities of nvcc's writing of the stick before any of it is put to the part (Q19)
`klq_decoder.cu`	a pair's verdict read back to its set and its concept, written beneath the pair
`carrier_flow.h`	a carrier's links read as what each writes, reads and loads, each register followed back to the operands of a case (P13)
`concept_product.h`	a pair's product, what its two forms make of what they read, held put by put and named a concept once it is whole (P13)

The rulesets `c.krs`, `cu.krs`, `pcie.krs`, `vhdl.krs` and `yosys.krs` are tracked here, and of the `.klm` `cu.klm` and `openqasm2_0.klm`; `Lstar.klq`, `sass.klm` and `sass.krs` the bridge writes as the SASS forms are learned. `orior protocol pairs pair` writes each pair's verdict beneath it.

The query. The crate `orior-query` at `src/ui/query` (the section "The query, a crate" below), `src/ui/cli/src/query.rs` its face on the command line, and an item of the Protocol menu in `src/ui/cli/commands.json`, the one list the command line and the window both read: `orior protocol query --<from> --<to> [--<part>]` names the language a program is written in, the language it is to be written in and the part it is to run on, each one the bridge's folder holds whole, a language its `.klm` and `.krs` and a part its `.khw`, `.kqr` and `.ksc`. It is one job of the catalog: the bridge written over the two rulesets (`write bridge`), then, where a part is named, the bridge's pairs put to the part (`pairs pair --part <part> --to <language>`). A name the folder does not hold whole is answered with each file missing, and nothing runs.

The checks. `utils/test/src/cu/transpiler/lstar/protocol/`, built and run by `utils/maint/engine/chain_check.sh`

file	holds
`query_ask_check.c`	the ask held to addresses whose answers are known (Q1)
`query_interface_check.c`	walks of asks from inside the interface held to known addresses (Q1)
`host_entry_check.c`	the kind and the width read from words put and given back
`bus_enum_check.c`	the classify and width asks, the region found, and the walk in probes
`gate_descent.c`	the gate run as the engine's own descent (Q4, Q15)
`chain_check.c`	how much of a relation the ladder's own cases decide (Q4)
`ask_order_check.c`	the known order held to its exact claims at every size (Q5, Q7)
`query_order_check.c`, `query_descent.h`	the known order put to the host, its run size and passes steered by the part (Q5)
`branch_side_check.c`	whether the side a branch is asked from leaves a mark (gnascor Open 4)
`stem_group_check.c`	the stem rule held to what it says (Q13)
`survivors_check.c`	the survivors a conjunction, no order of the cases changing which stand, and the descent's price (P2, Q4)
`answer_read_check.c`	the four answers read against the host's bracket, and gray where no answer decides (P11)
`scheduler_check.c`	the queue of gray entries, the most-settling ask first, carried in rounds (P13)

The record, checked. `utils/test/src/cu/types/file_defs/kqr/kqr_check.c`, built and run by `chain_check.sh`: R read whole and written whole, an ask it holds not put again (Q9).

The relations, checked. `utils/test/src/cu/engine/rmc/ruleset_read_test.cu`, run by `ruleset_read_test.sh`: every target's files and the map between cu's and sass's (Q19).

The part, asked. `src/cu/transpiler/vendor_bin_layouts/nvidia/scaffolding/`: the SASS probe (`interface_sass_probe_*.c`, `interface_sass_probe.h`), its runs (`interface_sass_*.sh`) and their records (`interface_sass_*.md`) (Q16, Q17). The probe goes through the toolkit and is scaffolding, an answer key and never the run channel.

The readings. `utils/maint/engine/`: `measure_check.py` (Q5, Q6, Q7), `order_check.py` (Q10 to Q14), `gnascor_read.py`, `gnascor_trace.c`, `gnascor_scenario.txt` and `gnascor_measure.py` (the state machine read off asks).

2.4 THE TABLE

Q1. The ask

THE ALGEBRA IT HOLDS TO `[address] -> (qualifier) -> [cost] -> bit`. The qualifier asks for a state validation and never a data payload. Over the bridge the address is a key of `Lstar.klq` read through a language's `.klm`, `[key] -> (relation) -> [cost] -> bit`, and

the qualifier asks one relation of the entries the key names. A relation of the text carries no cost, and its bit is 1, 0 or gray (P1).

DOES `src/cu/transpiler/lstar/protocol/teacher/run_channel.h` declares the channel: a container in, eight words put, four read back, and an outcome of answered, nothing, illegal or no channel. Nothing implements `run_channel_ask` and nothing calls it.

WANTS Every ask put with `host_put` and read with `host_read` (`src/cu/transpiler/lstar/protocol/query/host_entry.h`) and nothing between them and the part: no compiler, assembler, disassembler, object reader, vendor runtime or driver library. Where a part's addresses lie is itself asked, an address holding, fixed, live or answering nothing. The SASS probe under `src/cu/transpiler/vendor_bin_layouts/nvidia/scaffolding/` runs through the toolkit and is scaffolding, an answer key and never this channel. An ask at a key carries the key and the language it is read through, and its answer is written to the key's entry in `Lstar.klq`.

TRIED, AND WHAT IT GAVE **Proved** on the host (`utils/test/src/cu/transpiler/lstar/protocol/query_ask_check.c`, run by `utils/maint/engine/chain_check.sh`, 16 checks, 0 failed): `query_ask` (`src/cu/transpiler/lstar/protocol/query/query_ask.{h,c}`) finds memory the test owns holding and leaves it as it was, reads a written word as that word and no other, reads a word nothing writes as not advancing, and reads a bound with no clock as past the bound. **Measured**: the host's interrupt time at a fixed address advances under the ask, in steps of half a millisecond to a millisecond, and an ask clocked on it reads a cost of a whole step or nothing. A bound from it judges a run of asks. **Proved** from inside the interface (`utils/test/src/cu/transpiler/lstar/protocol/query_interface_check.c`, 8 checks, 0 failed): `query_interface_walk` (`src/cu/transpiler/lstar/protocol/query/query_interface.{h,c}`) runs `query_walk` in a probe and keeps an ending as the answer of the address that caused it. A read at address 0 ends the asker on an address fault, the page every Windows process shares answers 64 reads in one child and ends the asker on a put at each word, and of its first sixteen words the walk finds the two its layout names as counters, at 0x8 and 0x14.

STATUS **proved** (the ask on the host, the walk from the interface), **measured** (the host clock's step)

NEXT The run channel made of asks of this form (Open 1, after the algebra): the enumeration space found not by a written address but by its answer, a FIXED identifier beside a LIVE sizing register repeating at one stride, the kind from `host_address_ask` and the stride from `period_read`; a region's width read from the mask a LIVE register gives back when written all ones, the part setting it.

Q2. The unbound pass and the baseline

THE ALGEBRA IT HOLDS TO An ask with no bound returns its cost in place of a bit. The spread of those costs is `.knf`, and every bound after is expressed against it. Over the bridge a cost is kept at the key it was read at, its spread beside it, and a cost read in one language is read at the same key in another.

DOES `monolith_cost` (`src/cu/transpiler/vendor_bin_layouts/nvidia/monolith.cu`) reads the part's clock (`CS2R SR_CLOCKLO`) before and after a chain of each base precept, 100 steps a turn and `turns` turns, every step one PTX instruction under a predicate read from a parameter the assembler cannot see, and each precept again interleaved with every other. `monolith_run` puts it `cost runs` times and reads each section's least m, its spread s and its mean, and holds every chain's last word to `precept_applied` on the host.

WANTS A watchdog stop marked as a censored sample and not a cost. A reference ask put alongside each real one, in the same conditions.

TRIED, AND WHAT IT GAVE **Measured** on the RTX 3070, sm_86 (`monolith_run`, 10000 turns of 100 steps, 16 runs, 315 checks, 0 failed): every single-instruction precept reads m = 4.05 to 4.06 cycles a step, NAND and NOR 8.05, NOP and MOV 0. The spread s reaches 2.22 cycles a step alone and 2.53 interleaved, and the means, 4.14 to 4.45, sit inside it. A chain written plainly comes out of the assembler as one instruction a turn, an empty fence after each step notwithstanding; predicated, every step is kept.

STATUS **measured**

NEXT The spread written to `.knf` as the part's floor, and the bound Q3 reads off it.

Q3. The bound pass

THE ALGEBRA IT HOLDS TO The same ask with a bound returns 1 or 0. A missing address, a false qualifier, a timeout, an error and too many cycles all read 0, and what separates them is kept in the baseline and not in the bit. Over the bridge the bound of an ask at a key is read off the spread kept at that key.

DOES Nothing.

WANTS The bound read off Q2 and never written by hand.

TRIED, AND WHAT IT GAVE Nothing has run.

STATUS **theory**

NEXT Follows Q2.

Q4. The gate, as the engine's own descent

THE ALGEBRA IT HOLDS TO Each candidate arrangement is an alignment, the relation's cases are the needle, and the oracle answers whether arrangement (`corpus_at - needle_at`) gives case `needle_at` the relation's word. `anchor_steer_spawn_coarms` places the case that prunes most, stops where the best case prunes nothing, and leaves the survivors as its output. Over the bridge the candidates are the forms a language's `.klm` and the part's machine file give for a key, the cases are the key's in `Lstar.klq`, and the survivors are written back to the key. The reader's relations are a gate of the same form over a set of files: a set holding a witnessed absurd relation is refused, and no order of the files or of their lines changes which sets are refused (P2).

DOES `utils/test/src/cu/transpiler/lstar/protocol/gate_descent.c`, built and run by `utils/maint/engine/chain_check.sh`, over every relation the ladder holds, against `chain_build`'s candidates and its own 512 sweep words.

WANTS The descent run on the host before any target is asked, and only the cases it placed put to the target.

TRIED, AND WHAT IT GAVE **Proved**, every relation: the descent's survivors equal every candidate asked every case, and both equal what `chain_build` keeps with its sweep on (add 1,202 of 1,227, take 1,047 of 1,083, up 411 of 698, down 408 of 559, same 0 of 8), 0 lost. A descent with the destroy rule off leaves the same count as one with it on. **Measured**, what decides each relation: one full-width sweep word decides add, take and same, and up and down need a second whose count reads zero (`0x43f96f40`, `0x3ef25a20`). No case the ladder holds is among them. Every impostor the ladder lets through, 507 in all, dies at the first or second case placed. **Measured**, the planning: a level reads every case against every survivor, 924,931 to 1,888,145 questions per relation. It runs on the host for that reason. **Measured** on the part (`interface_sass_descent.md`): the cases `gate_descent` places for each relation, one or two, are put to every arrangement it descends over, 3575 of them, on the part alone. What stands on the part stands on the host for every relation, and no verdict differs.

STATUS **proved** (survivors), **measured** (deciding cases, planning cost, the placed cases on the part)

NEXT Put the descent ahead of every ask in the loop, and read its survivor count as the reading it is (Q11).

Q5. The order of asks for slicing

THE ALGEBRA IT HOLDS TO A chain of `n` links measured by asks that each cover half the links, any two overlapping on a quarter: ask `r` covers link `c` where `(r + 1) & (c + 1)` has an odd count of ones, the rows of a Hadamard matrix of order `n + 1` with its first row and column dropped. One ask informs every link at once. The per-link solve is `(n + 1)·x = 4·Sᵀb - 2·(Σb)·1`, integer adds and a division by a power of two. Over the bridge a link is a key: a construct's lines, each read through a language's `.klm` to its key, and each link's solved cost is kept at that key.

DOES `src/cu/transpiler/lstar/protocol/order/ask_order.{h,c}`: `ask_order_covers` gives the order from the link count alone and stores nothing, and `ask_order_solve` gives every link's cost scaled by `n + 1`. A count one short of a power of two, 3 to 63, has a known order, and every other count is refused and writes nothing.

WANTS A chain composed to a length a known order holds, and its asks emitted to a target (Q1).

TRIED, AND WHAT IT GAVE **Proved** (`utils/test/src/cu/transpiler/lstar/protocol/ask_order_check.c`, run by `utils/maint/engine/chain_check.sh`, 1,231 checks, 0 failed, at 3, 7, 15, 31 and 63 links): `4·Sᵀ - 2·J` times `S` is `(n + 1)` times the identity entry by entry; every ask covers `(n + 1)/2` links and any two share `(n + 1)/4`; integer link costs

solve back exactly at 1 to 8 passes; a fixed cost on every ask, and a cost growing with the square of the count an ask covers, each move every link by one amount and leave their order as it was; every count 0 to 127 without a known order is refused. **Measured**, against one ask per link at one budget (`python utils/maint/engine/measure_check.py`, check 4): the gain is (n + 1)/(2·√n), 1.09x at 3 links, 2.07x at 15, 8.16x at 255, against 1.15, 2.07 and 8.02 predicted. **Measured**, against a drawn order at the budget with nothing spare (check 5): the known order wins by 5.5x at the median worst-link error, 33.7x at the 95th and 125.1x at the worst of 400, and 17 of 400 drawn orders do not come apart at all. **Measured** on the host through the protocol (`utils/test/src/cu/transpiler/lstar/protocol/query_order_check.c`, run by `utils/maint/engine/chain_check.sh`): `query_order_put` (`src/cu/transpiler/lstar/protocol/order/query_order.{h,c}`) puts seven links, each a QUERY_ADVANCES ask at a word nothing writes and link k reading it 1 + 64·k times, on the clock the protocol finds by asking, read finely by counting reads of it between turns: each run's cost an exact rational, each of 64 passes solved on its own in exact integers, nothing rounded, summed or cut to a least. The run size is read off the part, the size whose weakest neighbor pair leans hardest; the floor against size has a least between the counting's spread and the interference long runs gather, and the size named moves from run to run (2^10 to 2^14 puts). Every neighbor pair reads dearer in 44 to 54 of 64 passes, past twice the spread. One read against five, QUERY_EQUALS against QUERY_HOLDS, does not order: an ask's own overhead is larger than four reads.

STATUS **proved** (the order, the solve and what the count's square does to it), **measured** (gain, known against drawn; the order put to the host)

NEXT Emit the order to the device once the run channel reaches its addresses.

Q6. Slicing by adjacent cuts

THE ALGEBRA IT HOLDS TO A chain's per-link cost read as the difference between neighboring prefix cuts.

DOES Nothing uses it.

WANTS Nothing: it is kept as the number that failed it.

TRIED, AND WHAT IT GAVE **Refuted** (`measure_check.py`, checks 1 and 2): a recovered link carries 1.43 floors of noise against a signal of 1.00 and orders 56% of link pairs correctly, where a coin orders 50%. 1,600 repeats of every cut order 95%.

STATUS **refuted**

NEXT Never tried again blind. Q5 replaces it.

Q7. The contention read

THE ALGEBRA IT HOLDS TO Where links contend the cost of a set is not the sum of its parts. Contention grows as the square of how many links an ask covers. What is left of a sweep

ask's cost once the known order's links are taken out is read against a constant, the count and the square of the count, and contention is read where the square's slope clears twice its own spread: `Puy²·(N + 1) > 4·Pyy·Puu` with the count taken out, every term an exact integer. Over the bridge the links that contend are named by their keys, and a contention read is kept with the keys of the asks it was read over.

DOES `ask_links_read` in `src/cu/transpiler/lstar/protocol/order/ask_order.c`. The sweep asks cover one link, half plus one and every link, in turn, each a prefix of an order of the links drawn from a seed.

WANTS Sweep asks emitted beside the known order's (Q1).

TRIED, AND WHAT IT GAVE **Measured** (`ask_order_check.c`, 15 links, 4 passes of the known order, 45 sweep asks, 300 trials, costs in integers around an overhead with a spread of 1000 a pass): contention found on 3.0% with none, 83.6% with up to 80 a pair, 100% with up to 200. **Measured**, the two terms beside the square: every ask pays an overhead, and the known order spreads it evenly over every link, which reaches a sweep ask in proportion to the count it covers. Read against the square alone the read found contention on 0% at every amount; with the constant and the count taken out it reads as above. **Measured**, where the sweep asks sit: covering every count from 1 to 15 found 54.0% at up to 80 a pair, and the two ends and the middle found 83.6% from the same 45 asks, the false alarms 2.0% and 3.0%. **Measured** in floating point (`measure_check.py`, check 6): an additive solve's per-link answers are 1.27x as far off at 0.08 a pair, and the square's term carried in the solve brings them back to 1.01x. On the host the sweep asks are put and timed as exact rationals (`query_order_sweep`), and the read itself takes integer costs: it is not yet put over exact rationals, and no host reading of it stands. **Measured** on the part (`monolith_run`, Q2): two chains with no link between them, interleaved a step at a time, cost the dearer of the two alone, in all 91 pairs of precepts that each cost a cycle a step or more. Chains that share no link do not contend.

STATUS **measured**

NEXT Emit the sweep asks to the device beside Q5's.

Q8. Cross-branch comparison

THE ALGEBRA IT HOLDS TO Two branches over one relation compared link by link through the signed per-link difference. The totals are even under swapping the branches and the question of which branch holds a link is odd under it. A total cannot answer it. The sign at each link is lead or rite, a difference inside the floor is dual, a link neither branch passes the gate on is void, and pass marks where the sign changes along the chain. Over the bridge two branches written in two languages are compared at the keys they share, each read through its own `.klm`, and a key one branch reaches and the other does not is gray at that link.

DOES Nothing.

WANTS A crossing at a pass costed as its own link, because a path that switches branches pays for the switch.

TRIED, AND WHAT IT GAVE Nothing has run. The rule that a symmetric magnitude cannot see an odd question is shown on the first move in chess (`examples/game_theory/6_oracle/first_move_advantage.py`).

STATUS **theory**

NEXT Follows Q5.

Q9. The record

THE ALGEBRA IT HOLDS TO A known order is a carrier and a shuffle is not: the same asks reach the part either way, and only the record of the order makes the answers decodable. The record holds every ask, its order and its seed, every answer with its cost, refusals and censored samples each marked, and then the winning path per problem over those same asks.

DOES `chain_shuffle` takes a seed. Every mode of `orior protocol pairs` writes the part's query record, `sm_86.kqr` (`src/cu/types/file_defs/kqr`, `kqr.c`), beside R: every untimed ask and what came back, each case's word or the refusal, timed_out where the watchdog ended it and censored where the gate held it off the part to protect the whole, or the carrier would not write its container or give each case a thread; every timed ask a sample of its own with the nanoseconds its launches took; then each pair and its verdict. A cycle reads it first and asks the part only what no cycle asked, a timed ask excepted and an ask that timed out asked again until an answer resolves it. An ask the part answered illegal, timed or not, is censored every time after without a process, the part's answer kept as it gave it. The asks and samples stand in the order they were put, each cycle that put a new one before them as `cycle <number> <mode>`, with `seed <seed>` where its mode draws one and `round <number>` in the pair mode; a cycle the record answers whole writes no line.

WANTS The query record, `.kqr`, under the member's stem beside `.kdm` and `.knf`, holding the order, the seed and the costs beside the asks (Open 3, after the algebra). The record stores what the host cannot reproduce and nothing it can. The known order follows from `n`, the sweep from its seed, the candidates from the ladder's cases and the descent's placement from those. The record holds those few numbers and the target's answers, and the file is small for that reason. `Lstar.klq` holds the questions put across languages and their verdicts; the record holds one member's asks and their costs.

TRIED, AND WHAT IT GAVE **Measured** (`pairs pair`, seed 1): the first cycle puts 384 asks and writes them to `sm_86.kqr`; the second answers all 384 from it, starts no process, and writes the same bridge and the same record. The curve of task 1006 (`pairs curve 1006`) writes 293 samples, 278 with the nanoseconds its launches took and 15 refused. The six loads the stall walk read too soon after `IMAD.WIDE.U32` wrote their address are answered illegal, and are censored in every cycle after.

STATUS **measured** (the asks, the costs, the order and the seed)

NEXT Follows Q1.

Q10. What an answer holds for

THE ALGEBRA IT HOLDS TO A reading answers for the part it was taken on and the size it was taken at, and for nothing else by default. Over the bridge the part and the size are carried in the key's entry beside each answer, and a verdict of the text, which no part gave, carries neither.

DOES Nothing records either.

WANTS The part and the size carried beside every answer. A generic block is the fallback for a member with nothing measured, never a result borrowed from one that has.

TRIED, AND WHAT IT GAVE **Measured**, on modeled costs (`python utils/maint/engine/order_check.py`, check 11): the arrangement winning at size 3 costs 79.4x the best at size 300, a winner carried to another part costs 3.06x that part's best, and the two penalties multiply.

STATUS **measured**

NEXT Carry both in Q9's record.

Q11. The survivor count

THE ALGEBRA IT HOLDS TO How many candidates survive the gate is itself a reading. One survivor decides the operator, and many say the cases do not, which another relation answers and no measurement can. Over the bridge the count is kept at the key, and an open pair's count of cases alike is the same reading (P10).

DOES Q4 reports it per relation.

WANTS Read in the loop and kept.

TRIED, AND WHAT IT GAVE **Measured** (`order_check.py`, check 8): 32 candidates over add fall to 11, 5 and 1 as cases are put. Q4 gives the same reading on the ladder's real candidates.

STATUS **measured**

NEXT Read it where Q4 runs.

Q12. One combined score

THE ALGEBRA IT HOLDS TO Precepts held and cost folded into one number at some exchange rate.

DOES Nothing uses it.

WANTS Nothing: it is kept with the numbers that failed it.

TRIED, AND WHAT IT GAVE **Not so** (`order_check.py`, checks 7 and 9): one score ships a wrong program at 2 of 5 exchange rates. A count of precepts held is not a count of independent facts either: 1 of 5 cases decides add on its own, and every case is then implied by the rest.

STATUS **not so**

NEXT Never one score. The rule at the top of this table holds it.

Q13. Membership by agreement inside a floor

THE ALGEBRA IT HOLDS TO Two members share a stem where their readings agree inside a floor.

DOES Nothing uses it.

WANTS Nothing: it is kept with the reason it fails.

TRIED, AND WHAT IT GAVE **Not so** (`order_check.py`, check 10): asked from one member's floor, agreement is not symmetric, and taken at the coarser floor it is not transitive. sm_86 agrees with sm_87 and sm_87 with sm_89, and sm_86 does not agree with sm_89.

STATUS **not so**

NEXT A representative per group, or a rule that builds the group and names its anchor (Open 5, after the algebra).

Q14. The readout

THE ALGEBRA IT HOLDS TO The bits a branch consumes resolve to a four-letter mnemonic per transition. Over the bridge a verdict reads 1 or 0 and an open reading reads gray, no answer, and a branch over keys reads gray where the bridge holds no verdict.

DOES `gnascor.md` holds the two transition tables.

WANTS Each name covering more than one transition says where its distinctions are kept, as the bound pass keeps them in the baseline.

TRIED, AND WHAT IT GAVE **Measured** (`order_check.py`, check 12, read from `gnascor.md`): of 49 transitions in the state table, 12 resolve to a name nothing else uses, and sync covers 22.

STATUS **measured**

NEXT sync split, or its distinctions kept in the baseline (gnascor Open 4).

Q15. One ask, many answers

THE ALGEBRA IT HOLDS TO An ask can carry many checks and its answer one bit for each. The channel answers in `RUN_OUT_WORDS` words of 32 bits, 128 bits an ask. A single bit over a set, 1 where every check in it holds and the set halved on a 0, carries less: it pays only where checks seldom fail, and past a failing share of (3 - √5)/2, about 38%, asking one at a time costs fewer asks. Over the bridge a check is a pair of a key and one case, and a pair a case has closed as two operations is closed on every set of cases holding that case and is not asked again (P5).

DOES Nothing emits a pooled ask.

WANTS A container that runs many checks in one launch and writes one bit each into its answer words.

TRIED, AND WHAT IT GAVE **Measured** (`gate_descent.c`, every arrangement against the cases Q4 placed): add, 1,227 checks with 25 failing: 1,227 asks at one a check, 10 at one bit a check, 129 at one bit a set halved on a 0 and 299 with the checks in a drawn order; take, 1,083 checks with 36 failing: 1,083, 9, 145 and 367; up, 1,396 checks with 538 failing: 1,396, 11, 1,593 and 2,141; down, 1,118 checks with 263 failing: 1,118, 9, 927 and 1,389. One bit a check takes the fewest on every relation. One bit a set takes more than one a check on up, where 38.5% fail, and its count turns on how the failing checks sit together.

STATUS **measured**

NEXT The answer carries a bit a check, and Q1's container is built to write one.

Q16. A form asked of the part

THE ALGEBRA IT HOLDS TO A slot of a ruleset is asked the way an operator is (P7). Every form the part's machine file holds is put in the slot with its operands filled by their kinds, the cases of the slot are put, and the forms that answer every case are timed and walked. The ruleset's form for the slot is the part's answer. Over the bridge the slot is a key of `Lstar.klq`, its candidates the machine file's forms, and the part's answer is written to the language's `.klm` at that key.

DOES `loop_back_if` is asked so: `interface_sass_probe` given `loop` and a machine file (`src/cu/transpiler/vendor_bin_layouts/nvidia/scaffolding/interface_sass_probe_ask.c`), run alone against the cubins of an earlier run through `SASS_PATTERN` in `interface_sass_test.sh`. **Measured** on the RTX 3070, sm_86, in two runs: 2533 of the 2927 forms assemble, 1715 fall through and 8 come back on every count, every one a `BRA`. No form answers 2 without answering every count. `sass_loop_walk` agrees with all 8. The 8 cost 33.7 to 34.7 ns a turn, and the next costs 0.0525 ns a turn more than the least against a spread of 0.9260: no form costs less (P6), and `sass.krs` keeps its `BRA`. **Measured** on the part (`interface_sass_writings.md`, `interface_sass_writings.sh`): every form of the machine file that writes a register from registers, predicates and numbers alone, 745 of 2928, is put in place of the kernel's IADD3, each through the gate, and run over 256 cases at once: 5685 cubins, 5394 run and the part refuses 291. Every precept that carries a word holds under at least one form, and every ladder relation but same. Every arrangement of `machines/sm_`

86.kdm, written node by node from the writings found, answers its relation on every case (interface_sass_chains.md): add 1202, take 1047, up 411 and down 408. Of sass.krs's forms for the word web's eight words of one precept, six are writings the part gives, and word_shl and word_shr are not (interface_sass_krs.md).

WANTS Every other slot a .krs writes by hand asked the same way, each with its own relation's cases, on every part with a machine file.

TRIED, AND WHAT IT GAVE The least cost read alone from one run: BRA.CONV in one run and BRA with a number in the next. Inside the spread the order is noise, and P6 reads it.

STATUS **proved** (the 8 forms, the walk), **measured** (their cost on sm_86, the writings and arrangements on the part)

NEXT Q17 says which slots to ask next.

Q17. Held against NVIDIA's compiler

THE ALGEBRA IT HOLDS TO A program written as C source and as the part's own code answers the same on every lane and costs no more than what NVIDIA's compiler writes for the C (P8). Where it costs more, the forms NVIDIA's compiler wrote over that run are the next slots Q16 asks. Its listing is a reading of what the part can do and never a rule: a form enters a ruleset once the part answers for it. Over the bridge the program's lane is read cu.klm to Lstar.klq to sass.klm, and a relation the map holds is a line the lane cannot be read through as it stands (P8).

DOES The monolith (src/cu/transpiler/vendor_bin_layouts/nvidia/monolith.cu): one program holding every base precept between tags, built once by NVIDIA's compiler, its listing the answer key. src/cu/transpiler/vendor_bin_layouts/nvidia/scaffolding/monolith_emit.sh holds every block against our reader, our assembler and the word our compiler writes the block's precept with through sass.krs, and writes monolith_differences.md whole on every run. src/cu/transpiler/vendor_bin_layouts/nvidia/scaffolding/sass_lane_needs.sh decides the 8 record programs the host oracle runs for SASS: 16 lane programs written, 84 of the schema's 99 forms asked, every instruction assembled against sm_86's machine file (936 for arithmetic, 1110 for division, 0 refused), and no lane run on the part. The same programs run as C source on the record tests (engine_table.md, item 11).

WANTS Each lane assembled into a cubin and run where its C route runs; both read word for word against the host; both timed (P6) and their instructions counted; every program where ours costs more read form by form against NVIDIA's listing.

TRIED, AND WHAT IT GAVE **Measured** (monolith_differences.md, 98 of NVIDIA's instructions over 18 blocks): where the machine file holds a form, our reader gives NVIDIA's text and our assembler its operation bits on every instruction. 1 has operation bits apart and 85 scheduler bits apart. **Measured**, each field the disassembler hides put to the disassembler a bit at a time: a field whose value renames the operation (IMAD's multiplier at bits 32 to 63, .

`MOV` at 0 and `.IADD` at 1), one whose 0 drops the operand (`BPT.TRAP`'s code from bit 34), the descriptor register a `term[URn]` address does not name, and an operand a form holds and does not print, whose run a sibling one bit away prints (`IMAD`'s carry-in at 87 to 90, in 685 forms). **Measured** on the part (`interface_sass_unprinted.md`: every value of each field written into a question's instruction and run over one case, no disassembler asked): `IMAD`'s carry-in without `.X` and `ISETP`'s predicate at 68 to 71 without `.EX` answer as printed at all 16 values. The 1 instruction still apart, `IMAD.IADD` with P2 at 87 to 90 where NVIDIA writes !PT, is apart only at bits the part does not read there. The load's bits 64 to 67 are one predicate, its index inverted at 64 to 66 with 000 for PT and bit 67 negating it, and a load whose predicate is false writes 0. Our assembler writes a printed load predicate there inverted, and the part answers P1, P2, !P1, !P2 and !PT as written. The descriptor register of the load (32 to 37) and of the store (64 to 69) is read whether bit 101 is set or clear: an odd register or UR62 is refused as an illegal instruction, and every even one below answers as printed whatever it holds. Bits 38 and 39 of the load and 70 and 71 of the store answer as printed at every value. **Measured** on the part over every form holding an unprinted operand (`interface_sass_unprinted_forms.md`, each form written with its operands filled by their kinds, four copies to a run): of 437 forms, 368 are asked, and 375 of their 379 runs answer alike at every value asked, every `ISETP`, `IMAD`, `LEA`, `HFMA2`, `F2FP`, `UIMAD` and `ULEA` among them. A kernel refuses a register number past the count it declares: `F2FP`'s run at 64 to 72 is refused at bits 68 to 71 under the 10 the pattern declares and answers alike at every bit under 255. None of the 40 atomics holding an unprinted register at 64 to 71 runs at its own bits: 22 are refused as illegal instructions, and 18 take a 32-bit or shared address nothing in the question backs. Every one was reached by turning bits; NVIDIA's compiler writes an atomic only for an atomic on `.global` through a 64-bit pointer, `ATOMG.E.ADD.64.STRONG.GPU` for `atom.global.add.u64`, `ATOMG.E.CAS.64.STRONG.GPU` for `atom.global.cas.b64` and `RED.E.ADD.STRONG.GPU` for `red.global.add.u32`. On its 64-bit add, run at every value of bits 64 to 71: 64 to 69 are the descriptor register, an odd one or UR62 refused and every other answering as printed whatever it holds; bit 70 clear is refused as an illegal address, and bit 71 set beside it as an illegal instruction. The pair R100 and R101 named there answers as printed at 0, at 4 and at 1 in its high word: no register is read from those bits. **Measured** on the part over every form `sass.krs` uses (`interface_sass_fields.md`, each operation bit turned and run, no disassembler asked): of 75 forms, 1619 bits read inside a run the machine file records and 526 outside every run, and a turned bit whose opcode holds a control transfer, a wait or no form is not run (P9). Our compiler writes AND, OR and XOR bit for bit, and JCC's guard and operation; SHL and SHR one bit apart, NVIDIA's `.W` against a clamp that answers 0 for a count of 32 or more where the precept wraps it; ADD and SUB as `IADD3` where NVIDIA writes `IMAD.IADD`; and for NOT, NAND, NOR, ASR, ROL, ROR and BRA it has no word. **Measured**, cost (P6): NVIDIA writes NAND and NOR as two `LOP3`, 8.05 cycles a step, where one `LOP3` is the gate, and ROL and ROR with the source's guard in front of the one `SHF` the part needs.

STATUS **measured** (the monolith and its record), **built** (the 16 lanes), **theory** (the lanes beside their C route)

NEXT Fix from the root up, each fix read off the record (Open 1, after the algebra): the machine file's missing forms, then `sass.krs`, then the word web.

Q18. A register's width

THE ALGEBRA IT HOLDS TO A register of the part holds w bits. A value of b bits past w is held in as many registers as it takes to cover b, its words in an order the part sets, and a value of fewer bits is held in the low bits of one. The width and the order are the part's own and differ from part to part, and the system classification routine asks both before any value is given a register. Over the bridge the width and the order are the part's `.kdm` entries, and a language's `.klm` keys a value to them and holds no width of its own.

DOES Nothing asks them. The measuring stick's reader holds 32 bits to a register and a value of 64 bits in an even register and the one past it, its low word in the even register (`src/cu/transpiler/vendor_bin_layouts/nvidia/measuring_stick_engine.cpp`).

WANTS Each asked on the run channel and held in the part's `.kdm`, by several questions that must agree: a register written oxFF, oxFFFF and oxFFFFFFFF and one past each, read back for where the word is cut; 1 shifted left until it reads 0, the count of shifts the width; a sum carried past the top, read for where it comes back to 0; and a value of one bit, such as ob1, put where a register is taken, which the part refuses and names the full width.

TRIED, AND WHAT IT GAVE Nothing has run.

STATUS **theory**

NEXT Asked in the system classification routine on the run channel; the reader then takes the width and the order from the part's `.kdm`.

Q19. Absurdity from a part of a set

THE ALGEBRA IT HOLDS TO A relation among entries is witnessed where every set holding its entries holds it, and open where it is a relation of absence (P10). No set is known to be whole: a witnessed relation is a verdict on any part of a set holding its entries, and an open one never is. Sameness read off the part is a relation of absence over cases: one case apart witnesses two operations, and alike on every case asked leaves the two names open, as P2's survivors stand. A map from one language to another is read through `Lstar.klq`, and of its relations a name one text with another on one side and two on the other, and a name given as another kind, are witnessed, and a name the other language does not give is open.

DOES `ruleset_core_relation.h` (`src/cu/types/file_defs/readers`) reads any files of rulesets, device maps and classifications together, each line an entry that names its relation on its own line, and keeps 18 kinds of absurd relation with the entries each names. `RULESET_CORE_RELATION_OPEN` marks the two of absence: a construct's line writing a name no entry gives, and a schema name no entry gives. `ruleset_read_test.cu` fails a set on a witnessed relation and writes an open one. `klq_identity.cu` (`src/cu/transpiler/lstar/protocol`, run by `orior protocol pairs`) reads nvcc's writing of the measuring stick: two questions

apart in one link a side are a slice whose two links are its identity, the carriers of an identity are gathered by their context in their own chains, each chain and each window of it is an identity at the first address it occurs at, and a candidate chain is sifted through those identities before any of it is put to the part.

WANTS Every word a language's parts make, put together by their categories with no part's meaning known, read through the language's `.klm` into the bridge's keys and held against these relations first: a word that holds a witnessed one is thrown out and asked nothing. Each word left is asked one case at a time in the descent's order (Q4), of nvcc's writing of the measuring stick's question whose C the word is, with the host computing that C on the case, until the part is asked through the run channel (Q1). The first case apart closes a pair as two operations, and a pair alike on every case asked is kept with its count of cases (Q11). A test and the select that reads it are one word, a part's operands are parts of their own category, and a word no question compiles to stays gray. Each verdict is written to `Lstar.klq` at the pair's key, each case in the case form of `.gsm`, and each language's operations read off the verdicts to its `.klm`. A text the same as another's and a construct that is one form renamed are witnessed by the text and put to nothing.

TRIED, AND WHAT IT GAVE **Measured** on the host (`ruleset_read_test.sh`, every target's files against its schema): sass 262 witnessed, ptx 65, c 25, cu 25, vhdl 9, yosys 0, and 0 open, every name a construct writes and every name a schema holds being given by a file its target reads. Files given on the command line are read the same way against no schema. **Measured**, the map read directly between two sets with no `.klm` written: cu's files to sass's hold 32 witnessed relations and 0 open, and sass's to cu's 32 and 0. **Measured** on the measuring stick (`orior protocol stick` and `orior protocol stick engine`, 1006 kernels, no device): of the 44 open pairs in sass's files, 15 are two forms writing the same operations apart in their operands, and 29 write different operations. nvcc writes both forms of each of the 29 in a question of the stick, and the engine both of 15. **Measured** on the stick's 1006 questions (`orior protocol pairs`, no device): 2972 slices carry 1007 identities in 1113 contexts. The host computes the 325 integer questions over 648 cases each, 351 contexts close on a case apart, 762 are asked of nothing, and rule 2 keeps 4 links off the part. The 1006 chains hold 679 whole identities and 2029 primitives and read as 12492 constituents. nvcc's listing sifted through its own identities is known whole 1006 of 1006. Of the engine's 320 chains none is known whole: 63 of their links nvcc never writes, and 1832 of their pairs side by side nvcc never writes so.

STATUS **measured** (the relations on the host, the identities, contexts, chains and sift on the stick), **theory** (the questions put to the part)

NEXT Put the primitives together by their categories, sift each through the chain identities, and add those nvcc has never written to the stick as questions, each marked with the rule that keeps it off the part where one does.

Q20. The run, a database

THE ALGEBRA IT HOLDS TO A run is a table for each area of it: the teacher's asks, the carrier's launches, the gate's verdicts, the overseer's conferences, the cross's pairs, the writers' files

and the traces. A row is fixed numbers: its key, the run's sequence; its who, the source, the line, the program and the time; and the columns its table names. A text is held once and a row names it by its number. A row names the rows it follows from by their keys, every relation one column cannot hold a row of the links table, and a question about a run is a walk over keys. A trace is rows: each carries the source and the line of the code that made it, and an area's trace is its rows in the order of their keys. A file is a view of the run, written in its place as the run ends.

DOES A run's `.qry` is one table of blobs, a row a blob (`qry.h`). Each process of a run hands its ordinary blobs into a ring of its own and the writer moves each ring into the run in turn, a panic and a critical blob going in ahead of every member (`qry_buffer.h`). A file handed to the run is written in its place once the run ends (`qry_filed`), and while the run runs every file is read from memory: a file the run reads from the disk is taken into the run the first time it is read (`qry_file_read`). `.kqr` holds the teacher's asks as records of numbers and their texts once each (`krep_records.h`, Q9). A k-file a run is to write is a view of a table of the run: its head, each text and each record it holds a row, every record put, given other words or let go after a row, and the file written as its rows put it together (`krep_tabled_by`); the teacher's R and `klq_identity`'s record are each a view of the table of asks (`kqr_tabled_by`).

WANTS A launch row and a pair row made on a run that reaches the part, and a tool outside the run pointed at a stream (The run, a database, after the algebra).

TRIED, AND WHAT IT GAVE A ring for each member against the one lock (`build/harness/qry/hand_time.sh`, 1, 4 and 16 processes): a hand of 256 bytes a median of 0.1 µs against 1.2 to 1.4, its 99th hundredth 1.4 to 2.3 µs against 5.5 to 18; a hand of 64 KiB a median of 31 to 41 µs against 30 to 50 and its 99th hundredth 170 to 2491 µs against 56 to 341; every byte in the run as handed. The rows of a record put together the record as held, every text and every record, on `kqr_check`'s own record before and after what a run puts and on the part's three records; the largest, 15023 records, is made a view in 77 ms and put together in 142 ms (`build/harness/kqr/table_time.sh`).

STATUS **measured** (the rings, the table of asks), **built** (files written as the run ends, every file read from memory while it runs, `.kqr` a view of the table of asks, the gate's verdicts, the carrier's launches, the overseer's accounts, conferences and lessons, the cross's pairs, the writers' files, the links and the table of texts, `kdecode` the select over the tables)

NEXT A launch row and a pair row made on a run that reaches the part.

2.5 THE ALGEBRA OF THE PROTOCOL

Every line is exact. A cost is a reading and carries a spread; a relation is arithmetic and carries none.

Every line takes the entries of the bridge as its inputs (The inputs, above). An address is a key σ of `Lstar.klq`, M_L is language L's `.klm`, which gives each name of L its key, and K(σ) is the key's

cases. A candidate is a form a language's map or the part's machine file gives for σ, and what a step finds is written back to σ.

2.5.1 *P1. The ask and its bound (Q1, Q2, Q3)*

An ask a = (address, qualifier) returns, unbound, its cost t(a). Bound by β it returns one bit:

bit(a) = [qualifier holds] · [t(a) ≤ β]

A missing address, a false qualifier, a timeout, an error and too many cycles all read 0. What tells them apart is the baseline, the spread of unbound costs, and the bound is read off it (Q2) and never written by hand. A run that never returns is apart from all of these, a reset and not a reading, and P9 holds it.

Over the bridge an ask is a = (σ, L, r): a key, the language whose map it is read through, and a relation r of the entries M_L gives at σ. A relation of the text carries no cost and no bound enters it. On a part E of a set of entries:

read(a, E) = 1 where r is witnessed and r(E); 0 where r is a relation of absence and not r(E); gray otherwise

A relation the part answers, forms f and g on the cases K(σ), reads as P1 reads each answer, its kind with it, so that a refusal from one where the other answers is a case apart:

read(a, K) = 1 where f(k) ≠ g(k) for some k ∈ K; gray where f(k) = g(k) for every k ∈ K, and |K| its reading

Neither ever reads 0: no answer of the part confirms two names one operation (P10).

2.5.2 *P2. The gate (Q4)*

A relation r is held on cases K = {k_1, …, k_m}. A candidate c survives when

c(k) = r(k) for every k ∈ K

- **The survivors are a conjunction.** S = {c : c(k) = r(k) for every k ∈ K} does not depend on the order of K, and no order of the cases changes which candidates survive.
- **The order is the price.** Putting K in order π costs Σ_c f_π(c) evaluations, f_π(c) the place of the first case c fails (m for a survivor). The descent puts first the case the most remaining candidates fail.
- **No cost enters S.** A candidate that fails a case is wrong on every part, and no part is asked about it.
- **Proved** (Q4): the descent's survivors equal `chain_build`'s on every relation the ladder holds.
- **Over the bridge.** With C(σ) the candidates of a key and r_σ its relation, S(σ) = {c ∈ C(σ) : c(k) = r_σ(k) for every k ∈ K(σ)}, written back to σ. The reader's gate over a set of files is the same conjunction: a set E is refused where R(E) for a witnessed absurd relation R, and is then refused in every set holding it. No order of the files or of their lines changes which sets are refused: no entry waits on another. A set not refused stands open, as a survivor of the gate does.

2.5.3 *P3. The order of asks (Q5)*

For a chain of n links, n + 1 a power of two, ask r covers link c where (r + 1) & (c + 1) has an odd count of ones: the rows of a Hadamard matrix of order n + 1 with its first row and column dropped, read as 0 and 1. With b_r the cost ask r returns and x the per-link costs:

$(n + 1)\cdot x = 4\cdot S^{T}b - 2\cdot(\Sigma b)\cdot 1$

- Every link is covered by (n + 1)/2 asks and any two links share (n + 1)/4, and that is the identity $4\cdot S^{T}S - (n + 1)\cdot J = (n + 1)\cdot I$, with $\Sigma b = ((n + 1)/2)\cdot\Sigma x$ (**proved** here for n = 3, 7, 15, 31 and 63). The solve is integer adds and one division by a power of two (**proved**, Q5).
- Against one ask a link, the gain is $(n + 1)/(2\cdot\sqrt{n})$, which $\sqrt{(n + 1)/2}$ approaches as n grows: each x_c takes every b_r at a weight of $\pm 2/(n + 1)$, and its squared spread is $4\cdot n/(n + 1)^2$ of one ask's (derived; **measured** 1.09, 2.07 and 8.16 at 3, 15 and 255 links against 1.15, 2.07 and 8.02, Q5).
- **Over the bridge.** The n links are keys σ_1, …, σ_n: a chain written in language L, each of its lines taken to its key by M_L. x_c is kept at σ_c, and two chains written in two languages compare at the keys they share (Q8).

2.5.4 *P4. Contention (Q7)*

Each of N sweep asks covers a count u of links, and y is what is left of its cost once the known order's links are taken out. The model is $y = \alpha + \gamma\cdot u + \delta\cdot u^2 +$ noise, and contention is $\delta > 0$. With every sum centered and scaled by N,

$C_{pq} = N\cdot\Sigma pq - \Sigma p\cdot\Sigma q$

and the count taken out of the square $s = u^2$ and of the leftover y,

$P_{ss} = C_{uu}\cdot C_{ss} - C_{us}^2$, $P_{sy} = C_{uu}\cdot C_{sy} - C_{us}\cdot C_{uy}$, $P_{yy} = C_{uu}\cdot C_{yy} - C_{uy}^2$

the square's slope P_sy/P_ss has squared spread $(P_{yy} - P_{sy}^2/P_{ss})/((N - 3)\cdot P_{ss})$, with N − 3 left over past the three unknowns. The slope stands above twice its spread exactly when

$P_{sy}^2\cdot(N + 1) > 4\cdot P_{yy}\cdot P_{ss}$

which is every term an exact integer and no division (`ask_links_read`, `src/cu/transpiler/lstar/protocol/order/ask_order.c`). Fewer than three counts covered leaves P_ss = 0 and nothing to read; $P_{sy} \le 0$ reads as adding.

Over the bridge the u links an ask covers are keys, and a contention read is kept with the keys of the asks it was read over.

2.5.5 *P5. One ask, many answers (Q15)*

A set of checks each failing with share p is asked as one bit, 1 where every check holds and the set halved on a 0. Past $p = (3 - \sqrt{5})/2$, about 0.382, asking one check at a time costs no more than any scheme that asks a set as one bit (cited from knowledge: P. Ungar, "The cutoff point for group testing", Comm. Pure Appl. Math. 13 (1960) 49–54). The channel's answer of 128 bits carries one bit a check, which pays on every relation (**measured**, Q15).

Over the bridge a check is a pair (f, g) at a key and one case k, and its bit is [f(k) ≠ g(k)]. A pair closed by k is closed on every K′ holding k (P10) and is not asked again: the checks still put at a key are its open pairs, each against the cases not yet asked of it.

2.5.6 *P6. The rank, read against its own spread (Q16, Q17)*

Each survivor c is run T times and its runs t_c,1, …, t_c,T taken. A run is lengthened by whatever else the host does and never shortened by it:

m_c = min_j t_c,j, s_c = max_j t_c,j − m_c, s = max_c s_c

With the survivors ordered by m, c_1 the least and c_2 the next, c_1 is the part's choice where

m_c2 − m_c1 > s

and otherwise the survivors are one cost and the ruleset keeps its form. The gate (P2) decides which candidates hold and this rule orders the survivors, and the two are never added together.

Over the bridge the survivors are S(σ), and the rank is kept at σ with the part and the size it was read on (Q10): the language's `.klm` gives σ the form c_1 where m_c2 − m_c1 > s, and otherwise keeps its form.

- **Measured** (Q16): 8 survivors in `loop_back_if`'s place on sm_86, m_c2 − m_c1 = 0.0525 ns a turn against s = 0.9260. The least of one run, `BRA.CONV`, is not the least of the next, `BRA` with a number, as a difference inside the spread gives.
- **Measured** (Q2, the monolith): 15 precepts alone, a million steps a chain, 16 runs. The eleven single-instruction precepts sit within 0.006 cycles of each other in m against s = 2.22, one cost. The two deltas that stand are 0 to 4.05, no instruction against one, and 4.06 to 8.05, one against two. `monolith_run` reads a delta inside s as dual or gray and one past it as lead (gnascor).

2.5.7 *P7. A form asked of the part (Q16)*

The slot is `loop_back_if loop where`. The question is a body the slot closes:

R7 ← 0, R0 ← N; turn: R7 ← R7 + 1, R0 ← R0 − 1, P0 ← [R0 ≠ 0]; then the candidate, guarded by P0

- A candidate that takes the way back on P0 runs N turns and answers R7 = N.
- A candidate that falls through runs one turn and answers 1.
- N = 2 separates the two, 2 against 1. The counts after it, 1, 3, 5 and 64, hold the answer to N: a form that came back a fixed number of times answers that number and not N.
- The candidates are every form in the part's machine file, with every register made R8 and every uniform one UR8, which the body does not keep, a predicate made P0 (PT, RZ and URZ kept), a label the body's and a number the distance back to it, 0x40 bytes over four instructions. Nothing decides beforehand which forms jump.

- A survivor is walked: `sass_loop_walk` reads its encoding back through the forms of the machine file and checks it as a loop's way back, the flag, the distance, and every register the body keeps left as it was.
- **Proved** (Q16): 8 of the 2927 forms answer every count, every one a `BRA`, and the walk agrees with all 8. No form answers N = 2 without answering every count.
- **Over the bridge** the slot is the key `loop_back_if`, its cases the counts, and the 8 forms that answer every count are S(σ), kept at it.

2.5.8 *P8. Held against NVIDIA's compiler (Q17)*

For a program p, C(p) is its text through `c.krs` and E(p) its lane through `sass.krs`. V(p) is what NVIDIA's compiler writes for C(p) and h(p) what the host oracle answers.

E(p)(k) = V(p)(k) = h(p)(k) on every lane k

is the gate, and the rank is P6 between E(p) and V(p), with |E(p)| and |V(p)| their instruction counts as a second reading. E(p) holds NVIDIA's line where m_E(p) − m_V(p) ≤ s.

- Where E(p) costs more, V(p)'s instructions over that part name forms, each in a slot. Each slot is a question for Q16: the forms of the machine file in that slot, the cases of the slot, and the survivors ranked by P6. NVIDIA's listing names the question and the part answers it.
- The loop closes when E(p) holds NVIDIA's line on every program p, and every form in every ruleset was answered by the part.
- **Over the bridge** E(p) is read through `Lstar.klq`: each line of p, written in cu's names, is taken to its key by M_cu and from the key to sass's names by M_sass, E(p) = $M_sass^{-1} \circ$ M_cu applied to p. A line whose key the map breaks, collapses or gives as another kind cannot be read through as it stands, and those are the map's witnessed relations (P10). A key sass gives no name for is open, and E(p) is gray at that line and not refused. **Measured** (Q19), the map read directly between cu's set and sass's with no `.klm` written: 32 witnessed relations each way, 0 open.
- **Built:** E(p) for the 8 record programs the host oracle runs, 16 lane programs, every instruction assembled against sm_86's machine file (Q17). **Measured:** V(p) for the monolith's 18 blocks, one base precept each, held against our reader, our assembler and E(p) instruction by instruction (`monolith_differences.md`, Q17). **Theory:** the rest.

2.5.9 *P9. The part's own bound (Q1, Q16, Q17)*

P1 counts a timeout among the readings of 0, which holds only where the run returns. A run that never returns is a reading apart: the part's display watchdog ends it with a reset, and across a run of asks that reset can hold a kernel's own watchdog past its bound and stop the host with a DPC_WATCHDOG_VIOLATION.

A body runs without end only where control reaches a line it has passed and the way back never closes. Four contexts reach it:

- A turned bit makes the instruction a branch to itself or a line before it. An isolated instruction turned this way is most often an illegal encoding the part refuses as a clean 0; a turned branch is not refused.
- The body holds a read it waits on, a register or a word read again until it carries a mark, and the turned bit closes the wait: the register the read names, the value it is held against, or the step that moves it. loop_back_if (P7) is such a body, its way back taken on Po = [Ro ≠ 0]; a bit that stalls the step or frees the branch leaves it turning without end.
- A barrier every lane is waited at, where a turned bit sends a lane down a path that never reaches it, and the lanes that waited do not return.
- The scheduler word waits on a scoreboard barrier a late result sets and that result never clears it. Above the operation the encoding holds a stall, a write and a read barrier and a wait over six barriers, none of it printed. A result ready after a fixed count carries that count as its stall; a result that comes back late, a load or a memory read, sets a barrier instead, and whatever reads the register it writes waits on that barrier. A barrier no instruction sets is at rest and a wait on it resolves at once; a wait on a barrier whose setter a turned bit sends a lane past never resolves, and the lane does not return.

The local absurdity, an isolated instruction turned illegal, the part refuses and the ask reads. The waited read is where a turned bit writes a lawful instruction that breaks the wait, and a lawful instruction runs where an illegal one refuses. An ask that holds a waited read carries its own bound in the body, a turn count past which no turned bit runs, and keeps that count in registers the turned bits do not reach: loop_back_if holds its candidate off the counter Ro, every register in the candidate made R8. The host's patience and the display watchdog are a reset, never the bound.

An ask that turns an instruction's bits is checked before it is put. The opcode, the low word's bits 0 to 11, names the operation, and a turned instruction is put to the part only where its key holds forms in the machine file and no control transfer or wait among them. Such an instruction falls through whatever its other bits hold. A turn past the key leaves the operation as it was. Every other turn is marked skipped and run on nothing, which closes the first context above for every key the machine file holds (`interface_sass_probe_fields.c`, Q17).

The scheduler word is set once for a run and no turned bit reaches it. The assembler writes every instruction to wait on all six barriers and to stall the soonest its operation's result is read, the longest where the krs measures no count, and to set a write barrier where the operation's result comes back late and a read barrier where it is a store (`sass_control_safe`). `cubin_safe` holds every instruction it reads to that stall. Which those are is the operation's schedule, a property of the operation the krs holds beside the scheduler's fields (`sass_operation_schedule`, `sass_machine`), and not of the one encoding the machine file first saw it with: that encoding sets no write barrier on `LDS` and no read barrier on any store. A turned bit lies at 0 to 104 and leaves 105 and up as they were: the safe word stands on every flip, and no flip opens a path around a barrier's setter. This closes the scheduler context for a flip probe, where the body is straight-line and every barrier set is cleared by the instruction that set it.

- **Measured** over the tree's CUDA sources built by NVIDIA's compiler for sm_86 (`monolith_scheduler.md`, 72 cubins, 96976 instructions, each named by our reader and its bits 105 to

127 read through the krs's fields, no listing read): NVIDIA gives a write barrier at every place to `LDG`, `LDS.64`, `LDS.128`, `S2R`, `S2UR`, `F2I`, `I2F` and `ATOMG`, and at most places to `LDS`, `MUFU.RCP` and `F2I.FTZ`; to 94 other operations at none. A barrier counts its producers, and a wait on it holds until every one of them is back: of 1716 first reads of a late result with its own barrier, a wait on that barrier stands before all 1716, at the reader or at an instruction before it. A late result NVIDIA gives no barrier is held behind another of the same operation out of the same unit: 20 first reads, 16 of `LDS` and 4 of `MUFU.RCP`, stand behind a wait on a later one's barrier, the results coming back in the order they were put; 6 of `MUFU.RCP` stand behind a wait on an earlier one's barrier and at least 16 cycles past it. No first read of a late result stands behind no wait. Every `STG`, `STL`, `ST`, `STS` and `RED` takes a read barrier at most or every place. A fixed result is first read no sooner than 4 cycles on all 21 integer operations timed over 67443 reads, 5 on `IMAD.U32`, `IMAD.WIDE` and `IMAD.WIDE.U32.X`, and 6 on `CS2R`. The safe word waits on all six barriers at every instruction, which holds every one of these. Six barriers on every part from SM75 on, each a count its producers raise and a wait drains, and loads that keep their order sharing one barrier, are as the published reverse engineering of the scheduling word gives them (cited from knowledge: the ptxas execution model at gh.evko.io/crucible-notes, and the control codes of the maxas assembler, github.com/NervanaSystems/maxas/wiki/Control-Codes).

- **Measured** on the part: a barrier set by an operation whose result is back in a measured count is never released, and a wait on it does not return. `sass_control_safe` sets none on such an operation, and `cubin_safe` refuses one that does (its rule 5). A pair a link writes and the next link loads through, read too soon, is read before it is back, and the load ends on an illegal address: the early read is legal, and the address it reads is none of the question's. Written by `IMAD.WIDE.U32`, the part refuses the load at stalls 1 and 2 and answers it from 3; written last by `IADD3.X` (`wide_add`) or `MOV` (`wide_pack`), at 1 to 4 and from 5 (`pairs stall`, every load the stick holds, 18 probes a writer). Every other pair read too soon answers apart and is never refused.

The gate (`cubin_safe`) reads a turn by its key, and a straight key is not a legal instruction: the key names the operation at bits 0 to 11, and the operation's own layout can refuse what the other bits hold. Where the machine file holds no form of a key, the gate passes a turn of it on the key alone, and an encoding only the operation's layout refuses falls through. Before a round reaches the part, each question is read against NVIDIA's disassembler off the part (`orior protocol cross`, `src/cu/transpiler/vendor_bin_layouts/nvidia`): the carrier (`cubin_run`) writes each container with the part taken out, the disassembler reads it back, and a question it calls an illegal instruction, or a control transfer at the slot, is held off the part on the live run however the gate read it. The disassembler names no operation and nothing it says enters a form: the held list is only the exact encodings not to ask. A round's forms are read in two passes, the turns of each form and then the register-runs its turns find, and neither reaches the part before the disassembler has read it (`orior protocol learn`). Each round is read as it is made, a round made from the answers before it as much as the first, and a question held only for a register past the count its kernel allots is put again with the count that clears it, which changes how it is put and not what it asks; each hold and each count given is a line of the teacher's trace (`run_channel.h`). **Measured** on the RTX

3070, sm_86: of the add's 93 turns one is held, the flip of bit 91, an illegal encoding the gate's key passed; the part learns the form with that one held and no reset (`khw_write`, kept in `sm_86.khw`).

2.5.10 *P10. Absurdity from a part of a set (Q19)*

A set of files E holds entries, and a relation R over entries holds of E or does not. R is witnessed where

R(E) ⇒ R(E′) for every E′ ⊇ E

and it is then a verdict on any part of a set that holds the entries it names. A relation of absence, that no entry of E gives the name x, holds of E and of no E′ holding a file that gives x. Read on a part of a set it is open: no read of a part decides it, and no set is known to be whole.

- **Witnessed:** a name given twice, the same text under two names, a construct that is one form renamed, names one word apart with texts the same but for their modifiers, a form opening another's literal operands, a construct that reaches itself, a parameter never written, a place no parameter takes, a form with no text, a register's numeric name where a fixed register gives it a name, a row given twice, a name the schema does not name, and a form taking another count of parameters than the schema says. Each is witnessed by the one or two entries it names. A classification's counts and classes are held within the file they are written in, which is read whole, and that file witnesses them.

- **Open:** a construct's line writing a name no entry gives, and a name of the schema no entry gives.

- **Asked of the part:** forms f and g put on cases K are two operations where f(k) ≠ g(k) for some k ∈ K, and that k witnesses it on every K′ ⊇ K. Where f(k) = g(k) for every k ∈ K, their sameness is a relation of absence over the cases, open on every K short of every case, and |K| is its reading, as the survivor count is (Q11). The text witnesses sameness and the part witnesses difference: no answer of the part confirms two names one operation, and no difference of texts confirms two operations.

- **The reading of a part:** a witnessed R reads 1 where R(E) and gray where not; a relation of absence reads 0 where the name is given and gray where it is not. A set known whole would leave no gray, and no set is known whole.

- **Maps:** the map from language L to L′ holds, for each name L gives, whether it is one text with another name in L and two in L′ (breaks), two texts in L and one in L′ (collapses), or given in L′ as another kind of form, nop, err or construct (kind). Each is witnessed by the entries it names. A name L′ does not give is open (absent). **Measured** (Q19): 32 witnessed and 0 open each way between cu's set and sass's.

- **The bridge:** each language is mapped once, to `Lstar.klq`, and a map between two languages is read through it, $M_L'^{-1} \circ M_L$: n languages take n maps, where read pairwise they take $n \cdot (n - 1)$. **Measured** (`orior protocol write bridge`, `cu.klm` and `sass.klm` with 151 keys each): every name of the 41 the map read directly between cu's set and sass's names is found on the two maps through the bridge, each way. Through the bridge 6 names more

each way are one text with another on one side and two on the other, where the other side gives one of the two as another kind, which the direct map reads as kind alone.

2.5.11 *P11. The ruleset, derived ask by ask (Q4, Q11, Q18, Q19)*

A ruleset gives each slot σ one form, and each fact of the part one number. Both are derived by putting relations to the part whose right side the host computes, one case at a time, each answer read as P1 reads it. No value is written by hand: a number enters as the answer of the host to a case or as the part's answer to an ask, and every walk over a number runs until the part says no.

The ask. A form f is put on a case k beside the answer of the host, h(k):

```
f(k) = h(k) ?
```

The part gives one of four answers. Alike, where f(k) = h(k). Apart, where it gives another word. Refused, where it ends the run as an illegal instruction, an illegal address or a launch out of resources. Nothing, where no answer comes back. Alike is truthy and the other three are falsy.

A case on which the C of the host traps or is undefined carries no h(k): a divisor of 0, the least value over -1, and a shift by the width or past it. The part's answer to such a case is read and kept, and gates nothing (`refused_<n>` in the host program `klq_identity` writes). Each is judged in the operand's own type: a `signed char` divisor put 0x100 is a divisor of 0.

What the host computes. Every integer question a slice holds, every task of register pressure, and every question the engine writes a chain for: each chain the part is asked is held to h on every case.

The cases. Each operand is put through 18 values: 0, 1, 2, 3, and the edges of every width, 0x7F, 0x80, 0xFF, 0x100, 0x7FFF, 0x8000, 0xFFFF, 0x7FFFFFFF, 0x80000000, 0xFFFFFFFF, 0x100000000 and the three edges of 64 bits. With a third operand of two values, a two-operand form meets $18 \cdot 18 \cdot 2 = 648$ cases.

The steps run in order, and each takes what the steps before it found.

Step 1. The width (Q18). Before any value is given a register:

```
read(put(0xFF)) = 0xFF ?  read(put(0xFFFF)) = 0xFFFF ?  read(put(0xFFFFFFFF)) = 0x
FFFFFFFF ? read(put(0x1FFFFFFFF)) = 0x1FFFFFFFF ?
```

The width w is the widest put that comes back whole, and the next one comes back cut at w. Once add stands (Step 2), two more asks must agree with it: x doubled from 1, `x ← x + x`, reads 0 at the w-th doubling and at no doubling before it, and `0xFFFFFFFF + 1 = 0 ?` is alike where w = 32. Where the three do not agree, nothing past this step is asked. **Theory.**

Step 2. Add, by the descent (P2). The candidates are every form the machine file holds in a two-operand slot, operands filled by their kinds. The descent puts first the case the most candidates fail. Five candidates and add, on four cases:

case	h = add	or	xor	and	take	mul
`1, 1 → 2 ?`	2	1	0	1	0	1
`0, 1 → 1 ?`	1	1	1	0	0xFFFFFFFF	0
`1, 0 → 1 ?`	1	1	1	0	1	0
`0xFFFFFFFF, 1 → 0 ?`	0	0xFFFFFFFF	0xFFFFFFFE	1	0xFFFFFFFE	0xFFFFFFFF

- `1, 1 → 2 ?` fails all five, and it is placed first: the carry out of the low bit is what add holds and no bitwise form holds.

- `0, 1 → 1 ?` and `1, 0 → 1 ?` fail a form that drops an operand.
- `0xFFFFFFFF, 1 → 0 ?` fails a form that saturates at the top, or carries into a 33rd bit.
- On the ladder's candidates the descent places one or two cases a relation, and every impostor fails at the first or second (**proved**, Q4). The survivors are S(add), and their count is a reading (Q11).
- At 2w a value is two words, and a form of 2w is a pair of forms of w. `0xFFFFFFFF, 1 → 0x100000000 ?` fails a pair whose two words are added apart (0): the high word takes the carry out of the low.

Step 3. Take. `1, 1 → 0 ?`, `0, 1 → 0xFFFFFFFF ?`, `1, 0 → 1 ?`.

- `1, 1 → 0 ?` fails add (2) and or (1); xor answers 0 and stands.
- `0, 1 → 0xFFFFFFFF ?` fails xor and or (1), a form that takes the operands swapped (1), and one that saturates at 0 (0).
- f(0, 1) ≠ f(1, 0) witnesses that the form does not commute, which add and every bitwise form do.
- At 2w, `0x100000000, 1 → 0xFFFFFFFF ?` fails a pair whose two words are taken apart (0x1FFFFFFFF): the high word takes the borrow out of the low.
- −a is 0 − a in a's type, put to take as `0, a`, and no form of its own is asked for it.

Step 4. Divide. `0, 1 → 0 ?`, `1, 1 → 1 ?`, `3, 2 → 1 ?`, `0xFFFFFFFF, 2 → ?`, `1, 0 → ?`.

- `3, 2 → 1 ?` fails a form that rounds (2).
- `0xFFFFFFFF, 2` is 0 signed, −1 over 2 cut toward 0, and 0x7FFFFFFF unsigned; a form that rounds down gives 0xFFFFFFFF. One case separates the three.
- `1, 0` and `0x80000000, 0xFFFFFFFF` signed carry no h(k). The part's word on each is kept at the key with the form that gave it. Two forms that give two words there are two operations (P10), and since C leaves the case undefined, either holds as the slot's form.

Step 5. Shift. `1, 31 → 0x80000000 ?`, `0x80000000, 31 → 1 ?` right, `1, 32 → ?`.

- `0x80000000, 31` right is 1 unsigned and 0xFFFFFFFF signed, which separates the logical shift from the arithmetic one.
- `1, 32` carries no h(k). A form that clamps answers 0, and a form that wraps answers 1. The case keeps the two apart as operations without gating either (Q17: NVIDIA's `.W` wraps, where the precept wraps too).
- Every case with an h(k) has a count under w, and its low word holds the count. A count held in two words is put as its low word, and every case with an h(k) answers the same.

- At 2w, `0x100000000, 1 → 0x80000000 ?` right fails a pair whose two words are shifted apart (0): the low word takes the bits the high word lets go.

Step 6. Compare, and the select that reads it. `0, 1 → 1 ?`, `1, 0 → 0 ?`, `1, 1 → 0 ?`, `0x FFFFFFFF, 0 → ?` for less-than.

- `1, 1 → 0 ?` fails less-or-equal (1).
- `0xFFFFFFFF, 0` is 1 signed and 0 unsigned.
- A test and the select that reads it are one word, asked as one (Q19).
- A bool is a relation and no narrowing: `(bool)v` is [v ≠ 0] over the whole of v. `0x100000000 → 1 ?` fails a flag of the low word alone (0), which a byte or a halfword is right to read.
- A flag read as a word and tested against 0 is the flag: [select(p, 1, 0) ≠ 0] = p on every case. The round trip is a cost to rank (P6) and never a gate, and a reader that does not hold it reads each conditional nested in another again for every reading of the one around it.

Measured, Steps 2 to 6 on sm_86: the engine's chain for every question the host computes, 441 of them, answers alike on all 648 cases and none apart. The 64-bit add, take, bitwise forms, not, negation and both shifts right are among them, each in the two words of NVIDIA's listing, every comparison glued to a second test by && or ||, and every select over a bool.

Step 7. Two forms, one slot (P10). Forms f and g in one slot are put on K. The first k with f(k) ≠ g(k) closes the pair as two operations, and k is kept at the key as its witness. A pair alike on every k ∈ K stays gray, with |K| its reading. A case closes a pair once, and a closed pair is not asked again (P5).

Step 8. The bound in place of a counterexample. An L* learner asks whether a conjecture is the machine and is given a case apart when it is not. The part gives no such case: K stands in for it, as a test suite does (learning_a_ruleset.md). An operation on words holds no state, and so the bound is on cases and not on states: a form is exact on K and gray past it. A two-operand form of 32 bits has 2^64 cases, and a form apart on a case outside K is found only by adding that case to K. **Measured** at 16 bits: 55 forms on all 65,536 cases against the integers of the host (learning_a_ruleset.md, Status).

Where the questions are bounded. Each step asks only what the step before it can hold, and each bound is a place the ruleset is gray. Read off the stick's 1016 questions:

- **The cases.** An operand meets 18 values and no other. Between 0x100000000 and 0x7FFFFFFFFFFFFFFF none is put, and a form apart only there is found by no case (Step 8).
- **The C of the host.** A case it traps on or leaves undefined carries no h(k), and the part's answer there is held to nothing (Steps 4 and 5).
- **The types of the host.** The host computes integers alone: 436 questions, and float and double, 398 of the 1016, are asked of nothing. No form of a floating slot is derived.

- **The reading of the engine.** A question the engine cannot write through cu.krs is asked of nothing: 166 besides the floating ones. 35 have no reading, 33 of them a quotient or a remainder, which sm_86's ruleset gives no form for (`err word_div`). 66 are calls or elements cu.krs gives no form for, 30 atomics and 34 statements.

- **The run channel.** A question carries as many cases as a launch gives threads, up to the `2^20` the host fills, and the run tool gives every case a thread. Every ask of the protocol puts all 648 cases the host computes: the register walk holds each probe to every one of them and reads the last register R252 over 35 asks, 9 alike, 20 refused and 6 held by the gate (`KLQ_TRACE` writes each ask and its answer).

- **The slices.** A context no two questions of the host hold is asked of nothing: 762 of 1113.

- **Two words.** A 64-bit form is put as two forms of 32 bits as NVIDIA's listing writes it, and the width that would ask it is Step 1, theory.

- **The walks.** The register walk starts below the top of the field, 255, and the threads stop at the `2^20` the host fills cases for (Step 9).

Step 9. The part's numbers, walked until it says no. These relations are over a number n and not a word. Each holds below a bound and is refused past it:

holds(n) ⇒ holds(m) for every m < n

and the walk reads one bound, the greatest n that holds.

- **The last register.** `holds(R_n) ?`: a chain register written and read back, renamed R_n, in a kernel declaring 255, and the control renames it to the least number nothing uses. The walk goes down from the top of the field below RZ, the name the field's top gives the register that reads 0. n = 254 and 253 are refused as illegal instructions, and 252 is alike: last = 252. **Measured** on the RTX 3070, sm_86 (`klq_identity register`, kept in `sm_86.ksc` as `run answers 000000fc register last`).

- **The launch.** `run(r, t) ?`: t threads a block, doubled from 1, with r registers declared. The part refuses past a bound at each t, and the walk reads it: on sm_86, task 1006 answers at every count of registers the walk put, up to 255, in blocks of up to 256 threads, at 128 and is refused as out of resources at 129 in blocks of 512, at 64 and is refused at 65 in blocks of 1024, and a block of 2048 threads is refused as an invalid value at every count (`klq_identity curve`, `curve.txt`). That bound is the part's answer and is written nowhere. The host bounds only the threads it fills cases for, `2^20` in all (`CUBIN_RUN_THREADS_MOST`, `cubin_run.c`). **Measured.**

- **The soonest read.** `alike(w, s) ?`: a chain where writer w's result is read by the very next link, every stall the longest and w's stall s. The stall runs the other way, holds(s) ⇒ holds(t) for every t > s, and holds at 0 on every pair the part answers: stall 0 is no zero cycles. Every stall from 0 to 15 of every pair is put in one round on up to three probes, and the soonest is the least s from which every longer stall answers alike. **Measured** on sm_86 (`pairs stall`, kept in `sm_86.ksc` as `run answers <s> stall <writer> <reader>`): integer, logic, compare and select pairs 4; `IMAD` into `LOP3.LUT`, `PRMT` and `SHF.R.S32.HI` 5; `IMAD.WIDE.U32` into `IADD3`

5 and into a load 3; `ISETP.GE.U32.AND` into `EXIT` 13; `IMAD.WIDE.U32` into `STG.E.64` and `IMAD` into `ISETP.GE.U32.AND` alike at every stall. A pair no chain of ours holds is probed in the arrangements of the `.kdm` that hold it, written at their operator's link: both rotates, `SHF.L.W.U32.HI` and `SHF.R.W.U32`, 4.

- **The fewest registers.** `alike(T, r) ?`: task T run with r registers declared. r is walked up from the count of registers the chain names until alike, and halved down until not, at one thread a block. A container declaring d gives code R0 to R(d − 3), as the last register gives at d = 255. **Measured** on sm_86: chains naming up to R21, R69 and R197 are refused at 22, 70 and 198 declared, and answer at 24, 72 and 200. A chain naming up to R13 answers at 8 and at 1: under some count the part gives a container more than it declares.

Step 10. The cost of a task, by the bounced sustain. c(r, t) is task T's cost on the part's clock, read over 100 launches, at r registers and t threads. The ask is

`c(r, t) ≤ hi ?`

against a band B = [lo, hi] read at the fewest registers:

- An ask inside B sustains it. B holds after 5 asks in a row land inside it without widening it.
- An ask past B is falsy only where it lands past B twice with the baseline asked between them landing inside B. Where the baseline strays, B widens to hold it and the count starts again.
- knee(t) = the least r with c(r, t) past B, found by bisection over r, which reads one bound where c(·, t) past B holds of every r above it.
- Each t is walked, doubled from 1 until the launch is refused (Step 9), and the curve of a task is its knees against t. **Measured** on task 1006 at one thread: 1.80 ms up to r = 128, 2.11 ms at r = 129, 3.04 ms at r = 192.

Step 11. The ruleset. For each slot σ the survivors S(σ) of Steps 2 to 7 are ranked by P6, and the slot is given c_1 where m_c2 − m_c1 > s and keeps its form where not. Steps 1, 9 and 10 give the part's numbers: the width, the last register, the bound on a launch, and each task's knees. Each is kept with the part and the size it was read on (Q10), in the part's `.ksc`.

2.5.12 *P12. Categories, and the coherence set C* (P10, P11)*

The ruleset L* of one known language is finite: the language holds finitely many forms, and P11 decides each on finitely many cases. The set of all of them is not finite:

L_all = {C*, L_1, L_2, …}

C* is our coherence. It is made of the categories we understand, each a pool of questions split from the others by what its questions ask. Our coherence has limits, and C* runs past them toward the edge of human coherence. Where we have no category, the system answers for itself.

set	category	meta
N_Lɣ	negation	core logical & semantic operations

set	category	meta
O_Lɣ	operation	core logical & semantic operations
C_Lɣ	comparison	core logical & semantic operations
L_Lɣ	qualifier	core logical & semantic operations
Y_Lɣ	causality	core logical & semantic operations
CO_Lɣ	concession	core logical & semantic operations
J_Lɣ	control	execution & state control
W_Lɣ	switch	execution & state control
S_Lɣ	statement	execution & state control
U_Lɣ	unknown	execution & state control
X_Lɣ	exception	execution & state control
D_Lɣ	descriptor	attributive & data elements
M_Lɣ	modifier	attributive & data elements
R_Lɣ	range	attributive & data elements
V_Lɣ	vector	attributive & data elements
K_Lɣ	quantity	attributive & data elements
G_Lɣ	timing	temporal & scheduling actions
H_Lɣ	schedule	temporal & scheduling actions
Z_Lɣ	sequence	temporal & scheduling actions
B_Lɣ	verb	metadiscourse & linguistic scaffolding
ME_Lɣ	meta	metadiscourse & linguistic scaffolding
P_Lɣ	punctuation	metadiscourse & linguistic scaffolding
A_Lɣ	anaphora	metadiscourse & linguistic scaffolding
DE_Lɣ	deictic	metadiscourse & linguistic scaffolding
EG_Lɣ	exemplification	metadiscourse & linguistic scaffolding
CL_Lɣ	clarification	metadiscourse & linguistic scaffolding
T_Lɣ	transition	interaction & context management
I_Lɣ	input	interaction & context management
Q_Lɣ	query	interaction & context management
F_Lɣ	feedback	interaction & context management
E_Lɣ	evaluation	interaction & context management
EM_Lɣ	emphasis	interaction & context management

A category. A category c is a pool of questions Q_c with its own cases K_c, its own answerer h_c, and its own bounds. h_c is the host where the host computes c, and nothing where it does not. The pools split the questions, each question in one pool:

$Q = \bigcup_c Q_c$, $Q_c \cap Q_d = \varnothing$ for $c \neq d$

Each pool is learned as L* learns a language (P11): an ask a case, and K_c in place of a counterexample.

Open-ended. No set of categories is known whole (P10). A category added is a pool asked as the others are, and a question no category holds is kept gray, never thrown out. The flow keeps the unknown.

Relational: never wrong, never whole. Every question at its root is a relation over forms and cases. A witnessed answer holds on every set holding the entries it names (P10). A category, a case or a language added never undoes it, and a concept read so is never wrong. A relation of absence stays gray on every part, and no reading is whole, ever.

The language bounds itself to the answerer's rules. A form enters a ruleset only where the part answers it (P7, P8). What the part refuses, what it never answers and what no category asks stay gray. The ruleset is the answerer's rules as far as the questions reach.

We fill part of C*. The filled part is every category with an answerer and a reading. We build to fill more: each form the part answers is a form to build the next question with (P8).

Measured, the stick's categories on sm_86. Each category is a pool. A question is written where the engine reads it through cu.krs, held where the host computes it, and alike where the part answers it as the host does on all 648 cases:

category	questions	written	held	alike
operator	171	131	131	131
conversion	110	72	72	72
test	90	90	90	90
compound	88	64	64	64
single math	87	0	0	0
double math	86	0	0	0
unary	78	64	64	64
casting intrinsic	71	0	0	0
single intrinsic	41	0	0	0
integer intrinsic	37	0	0	0
atomic	33	0	0	0
double intrinsic	28	0	0	0
statement	19	1	1	1
warp	17	0	0	0
memory	13	0	0	0
built-in	13	11	0	0
conditional	11	9	9	9
pressure	10	10	10	10
sync	9	0	0	0
call	4	0	0	0
all	1016	452	441	441

Every bound of P11 is a row of this table. A category with no answerer, the floating ones and the intrinsics, holds nothing. built-in is written and not held: the host computes no built-in. None is apart. Every question held answers without error, and every other question stays gray.

A floating category's answerer. The host gives a floating question no single answer, only what exact arithmetic gives. For two operands of width w, the exact value v of the operation is computed on exact numbers, and its two neighbors of width w bracket it, down(k) ≤ v ≤ up(k). The ask is a relation over the bracket:

```
f(k) ∈ {down(k), up(k)} ?
```

- A part's answer outside the bracket is apart, a wrong operation on every part.
- Inside it, which end the part takes is the part's own rule. Read across the cases, the ends it takes give its rounding, to the nearer with ties to even or toward 0, and no rounding is written by hand.
- A v the bracket holds exactly, down(k) = up(k), is a case with one answer and gates as an integer case does.

- The exact double is a mantissa and an exponent, m · 2^e, each lane its own exponent. Its sum, difference and product are exact, and its cut to width w keeps both ends (`edouble_record`, `src/cu/types/integerfloats/edouble`).
- **Theory.** The floating categories hold nothing until the stick's floating questions are put this way.

2.5.13 *P13. The sets, open-ended, and what they collapse (P2, P5, P10, P11, P12)*

Every set below is open. No set is known whole (P10), and an entry added is asked as the others are.

- **The categories.** C* = ⋃_c c, and a category is a pair c = (asked, answerer). The asked side is gnascor's thing and doing words, the tables of gnascor.md, and the sets of our coherence below, which those tables file among their thing, doing and gluing words. Each such set is named `<n>_coherence`, its stem what it holds. The answerer side is one of the kinds below. Each category has a clear operational line, and where two categories blur, the blur is another category.
- **The tree.** The sets are nodes of one tree of linked nodes. Each node is one equality test, and each set is the node of everything beneath it, a set of sets where it holds others. A node with more than two members is a binary tree in the first-child, next-sibling form: the first link goes down into the set, and the next goes to the member beside it. A sibling link between two members is their negation. A fact one node holds and several parents read, the statement, is one node linked from each, and its answer is written there once. A fact of more than one part is a set of its own, each part a member: a set that holds two kinds of thing is a clump, and it splits into a set for each. A set needs no name: its relation is its identity, the concept below. The metas are internal nodes: each says why an entry sits where it does, a pair hangs from every meta its tests read, and each is written above the set the pair is read into. The sets are our coherence, and a language is read through them by its relational map to them alone. `unknown_coherence` ends every set: an entry no answer has read further down stays there, a member of every set beneath. Read on this tree, the 25 pairs one text but for a modifier are 15 edges of the statement, 6 edges across the line from an equality to an order, 2 edges within order, swap and strict against inclusive, and 2 duals. The decoder reads all 15 into `statement_coherence`: two readings of the same bits under two statements agree wherever every value the link reads fits every statement, below the top of a byte read signed, and a pair alike on every such case and apart past them is the statement's. The values are those the part holds at the link, and a put with no read of them is read into no set.

```
equality_coherence              every ask: f = g over K
├─ concept_coherence            a product whole, hashed: one concept whatever names it
│  └─ unknown_coherence
├─ structural_coherence         a meta: one form the other with a parameter bound
│  ├─ bound to RZ               zero is eq at RZ, negative is lt at RZ, mul is mul_add at
↪  RZ
```

```
│  ├─ bound to a literal          word_from_byte is word_bitand at 0xff
│  └─ unknown_coherence
├─ syntactic_coherence            a meta: two operations, one meaning written twice
│  └─ unknown_coherence
├─ intent_coherence               a meta: ours, imposed, the patterns we write in
│  ├─ literal                     a value a form writes of its own: 0x20, the warp's size
│  ├─ carrier                     the thread's index, the case's index, the bound, the
↪  addresses
│  └─ unknown_coherence
├─ pragmatic_coherence            a meta: a meaning the launch gives, y and z of an index
↪  or an extent
│  └─ unknown_coherence
├─ data_quality_coherence         a meta: answered, and the cases cannot make the product
↪  whole
│  └─ unknown_coherence
├─ noise_coherence                a meta: a row both alike and apart, decided outside the
↪  form's reads
│  └─ unknown_coherence
├─ folding_coherence              internal: our coherence held against itself
│  └─ unknown_coherence
├─ comparison_coherence
│  ├─ equality_coherence          eq, ne, zero, nonzero: no statement
│  │  └─ unknown_coherence
│  ├─ order_coherence             lt, le, gt, ge, negative: a statement
│  │  ├─ strict, inclusive        lt and le, gt and ge: apart at a = b alone
│  │  ├─ operand swap             lt(a, b) = gt(b, a)
│  │  └─ unknown_coherence
│  └─ unknown_coherence
├─ operation_coherence            a value computed: shift, add, multiply, bitwise, at any
↪  width
│  └─ unknown_coherence
├─ commutative_coherence          one value with the first two sources either way round
│  └─ unknown_coherence
├─ verb_coherence                 a doing word: load, store, move, copy, a special register
↪  read
│  └─ unknown_coherence
├─ statement_coherence            one node, linked from thing words, modifiers and orders
│  ├─ sign                        signed_, .S8 and .U8, SHF.R.S32 and .U32, ISETP .U32
│  ├─ width                       byte, halfword, word, wide, .64, .128
│  └─ unknown_coherence
├─ qualifier_coherence
│  ├─ memory promise              .CONSTANT
│  ├─ flag join                   _and, _or
│  ├─ guard                       if, unless: linked to the control's condition
│  ├─ exclusion                   atomic
│  ├─ scope                       STRONG, GPU, SYS, CTA, SM
│  ├─ cache                       EF, EL, LU, NA, LTC64B, LTC128B
│  └─ unknown_coherence
├─ modifier_coherence
│  ├─ dual                        and, or
│  └─ unknown_coherence
```

```
├─ negation_coherence
│  ├─ bitwise                    not
│  ├─ arithmetic                 neg
│  ├─ predicate                  !, unless
│  └─ unknown_coherence
├─ witness_coherence            known at a third form's link: both forms answered there,
↪  alike or apart
│  └─ unknown_coherence
├─ range_coherence
│  ├─ halves                     low, high
│  ├─ chain position             first, middle, last
│  ├─ sides                      left, right
│  └─ unknown_coherence
├─ vector_coherence
│  ├─ level                      thread, block, grid, warp
│  ├─ position, extent           idx, dim
│  ├─ axis                       x, y, z
│  └─ unknown_coherence
├─ timing_coherence
│  ├─ stall                      the soonest a result is read, a barrier and a wait,
↪  .reuse
│  ├─ operation                  each form's time, the curve's knees (Step 10)
│  ├─ vector                     the delta of a register, a lane, an operand of a case
│  └─ unknown_coherence
├─ control_coherence
│  ├─ transfer                   exit, return, a branch
│  ├─ loop                       the way back to a loop, the loop's label
│  ├─ condition                  if, unless: one node, linked from the qualifiers' guard
│  ├─ target                     label
│  ├─ dispatch                   a state's next, state_dispatch, state_next
│  └─ unknown_coherence
├─ switch_coherence
│  ├─ structure                  open, body, close
│  ├─ declaration                declare_*
│  ├─ note                       note
│  └─ unknown_coherence
└─ unknown_coherence
```

- **When a language is read.** A language is read through our coherence where its program builds: written from the rules the query has read, it answers alike with the host on every case. `unknown_coherence` empty at every node the program's path passes is the same condition. Where the program does not build, either the question was malformed, the part refusing the ask as it was written, or an entry the program needs is still in `unknown_coherence`, and the query runs another cycle. L* stays open: no set is known whole (P10), a language added brings its entries in at `unknown_coherence`, and a program built closes the path it reads and no other.

- **Every cycle ends.** A cycle ends in one of three ways, and each is bounded. An entry moves down a node, and a path of the tree is finite. A question comes back malformed, refused, nothing or held, and its path stops with the part's reason. A built program answers apart at

a case, and that case moves an entry: R holds every ask, and no ask is put twice. The cases K_c grow to test our coherence, each added by the descent (Collapse 4), and a run grows only by the cases added. Where a program does not build, the query says which of the three stopped its path: the question to put otherwise, the node still in `unknown_coherence` to ask next, or the case and the link the program answered apart at.

- **Folding.** `folding_coherence` is internal: our coherence held against itself. Where the tests grow past what the cycles settle, the tree is read for tests that answer one another, and each is factored out and asked once. One ask at a node answers both members a sibling link joins, its negation the other. `lt` answers `gt` across the swap. The statement is asked once at its node, and every parent linking it reads that answer. `zero` is `eq` at 0. What folds is the testing across nodes, and no entry and no answer is lost to it. The decoder writes `folding_coherence` above every pair whose concept another pair shares: one ask answers both.

- **Every set.** Our coherence is these sets, `equality_coherence` the root of all of them. The decoder writes `qualifier_coherence`, `modifier_coherence`, `statement_coherence`, `negation_coherence`, `witness_coherence` and `unknown_coherence` as the sets a pair's answers read it into, and `comparison_coherence`, `equality_coherence`, `order_coherence`, `operation_coherence`, `commutative_coherence`, `verb_coherence`, `range_coherence`, `vector_coherence`, `control_coherence` and `switch_coherence` as the categories its two texts read it into before any ask, all beneath its verdict in the order of the tree. `concept_coherence <hash>` and `intent_coherence <value>...` come beneath them, and `structural_coherence`, `syntactic_coherence`, `pragmatic_coherence`, `data_quality_coherence`, `noise_coherence` and `folding_coherence` above them as its metas. A text settles nothing: a pair the texts read into a category and no answer reads into a set is in that category's `unknown_coherence`, and is written with both. `timing_coherence` is named, with its line below, and no test writes it yet: its tests are the timed asks. A set grown from answers and named by no word is held by its concept. An entry is a member of every set and every meta a test reads it into, each membership a light of its own on the switchboard: a pair read into two sets is written beneath both, never folded into `unknown_coherence`, and is in that set only where no test reads it into any.

- **The concepts.** `concept_coherence` holds the concepts, each a product: whether the two forms write one value or two on each case, as a function of the values the form reads there, alike, apart or both at each combination of them, written beneath the pair as `concept_coherence <hash>` (`concept_product.h`). The values are the part's own, read at the form: the carrier is cut just before the form for each value it reads, and just after it for each form's product, and the value stored in place of its answer. A concept cares for nothing that made it. The links of a carrier before the form and after it are deltas of their own, and a product read through any carrier is the same where the values are those the cases are made of. A product is a concept once it is whole, a row at every combination of those values, a register and its `.hi` one wide value: a part of a product is not yet any one concept, and the query puts the pair on past its first case apart, at each link that reads the form in a shape no link read before, until the product is whole or every branch is put or steered off. The links are walked in bands of one delta, the nearest first, and each band is a steer: truthy where

a put in it adds a row that came back one way, signal, and falsy where its puts add none or only rows that came back both ways, noise. A truthy band steers on to the next, and a falsy band past the first case apart steers off every farther one. The cycle loops while the product is not whole and a branch is left, and ends where it is whole, where the question is malformed, or with every branch put. A product the question's cases leave short of whole is put at each new shape over every combination of the values the cases are made of, all three operands through them, run by run. The form's reads are taken in the order it names its parameters, a parameter the chain binds to a literal read as that literal, so that every carrier of the form reads one set of values; a parameter inside `[` and `]` says where the form reads and is none of them. Two pairs of one product are one concept whatever names them and whatever language wrote them, and a word, where one is given, is a key on the identity as a language's `.klm` keys its names to the bridge. `signed_word_shr` is one such: shifting a signed word right is a concept, whatever chain holds it.

- **The rounds.** Each pair is put in a thread of its own, and the pairs take turns in the order of the bridge, each running until it asks or is put. Once every pair left has asked, the round's asks go to the part together (`run_channel_ask_many`): the mundane in one process, and each dangerous one, a stand-in that reads or writes memory through an address or sends control elsewhere, in a process of its own. A mundane ask the part refuses ends its process, its ending is its answer, and the asks after it are carried again. No ask of a round waits on another's answer, since a pair asks its next only once its last is answered, and what depends on an answer is asked in a round after it. A round is a pass, and the rounds loop while a pair is left to put. Every mode plans its questions in tiers the same way: each question that waits on no answer but the tier's before it is written before any is put, and a tier is one round. `cost` is one round, every arrangement's check and its clock and each link's clock alone (6140 questions); `stall` is two, every chain at the longest stalls, then every stall of every probe of every pair (5058 questions, 43 s with the record answering what it holds). A round is carried 1024 questions to a process, for the memory each holds while it is put. Every answer is held by its question's identity, its code, registers, shape, launches and cases, in the part's query record, `sm_86.kqr`, and an ask of an identity held is answered from there, in this cycle or any later one: the earliest identity of a chain is asked once and read from then on. The record ends with each pair and its verdict, and an open pair is the question a further pass takes up.

- **The choices.** Every choice a put makes at a link is written to the log as `choice <flags> <link>`, the flags of the trace's table (`src/cu/types/file_defs/klq/query_trace.tsv`) that hold there ORed into one word: a link nothing reads after, the carrier's own, written as it stood, not assembled, steered off, of a shape read before, alike, apart or refused, and the put's own end, a const stop, a parameter of the stand-in with no argument, a stand-in the ruleset cannot write, or a form at no link. The table is the only place a flag is named: `klq_identity` and `klq_decoder` both read it, and the decoder writes the choices of every put and, for each pair the part answered nothing of, how many of its links held each flag, the question a further pass takes up. **Measured**, seed 1, the twelve pairs asked nothing: `thread_idx_x` and `block_idx_x` stand at 441 links, every one the carrier's own, and their `y` and `z` forms and every extent at no link; `wide_pack` and `wide_from_word` at 22, all the carrier's own;

`wide_mul_word_add` against `wide_mul_word` at 882 of the carrier's own and 10 the stand-in writes as they stood; and `test_wide_ne` against `test_word_nonzero` gives a parameter no argument either way round.

- **The lower witness.** A pair whose puts asked nothing either way round, neither form standing at a link that carries the cases, is put at the links of a third form: the first of the bridge's keys whose parameters name every parameter of both and that stands at such a link (`witness <form> <form> <third>` in the log). Both forms stand in at each of its links with the arguments its link gives by name, the part answers both over the cases, and the two answers are held against each other: f and g on K are two operations where f(k) ≠ g(k) for some k (P10). A pair the lower witness answered, alike or apart, is known: the decoder reads it out of `unknown_coherence` into `witness_coherence`. The first case apart closes the pair, written `closed <link> <case>-><first>/<second>`; alike on every case leaves it open with its count of cases. A link of a shape read before is passed over, and so is one whose writing reaches nothing the part answers, where both stand-ins answer as the chain did as it stood. Two forms that read nothing but the register they write answer the same at every link, and the first link left answers them. A parameter the third form's link gives no argument takes one through the pair's binding, what the other form writes in its place, and two stand-ins the binding writes as one text ask nothing. **Measured**, seed 1: ten of the twelve pairs asked nothing are put at `word_add` or `word_mul_add`, nine close and one stays open alike. `wide_pack` against `wide_from_word` is put at `word_copy`'s links through its binding, `low` the other's `from` and `high` `RZ`: at the 4 links that reach what the part answers it writes as `wide_from_word` writes, and it stays open, since only a link that gives a `high` other than `RZ` parts them. `test_wide_ne` against `test_word_nonzero` names parameters no third form gives, and no binding joins them.

- **The intents.** `intent_coherence`, a meta: our intent, imposed. We know the patterns we write in, and a value a form writes of its own says what the form is for: `MOV R12, 0x20` writes the warp's size. The carrier's own links, the thread's index, the case's index, the bound and the addresses, are read off the shape every carrier of ours has, and are intent of the same kind. The values a pair's two forms write as the ruleset gives them are written beneath it, `intent_coherence <value>...`, and each is put to the pair's product with every case value: the warp's size against a copied word agrees only where the word is 0x20, and the pair is a qualifier's, 0x20 its qualifier. A value the chain binds into a form's parameters is the chain's and enters no concept. Intent steers where the query looks and settles nothing: an entry stays in `unknown_coherence` until an answer reads it down, and since that set is a member of every set, intent imposed never poisons it.

- **The structures.** `structural_coherence`, a meta: one form is the other with a parameter bound. The two texts are read token for token, each parameter a marker, and where one writes a literal in place of a parameter of the other and every other token is the same, the two are one structure: `test_word_zero` is `test_word_eq` with its right operand `RZ`, `test_signed_word_negative` is `test_signed_word_lt` at `RZ`, `wide_from_word` is `wide_pack` with its high word `RZ`, `word_from_byte` is `word_bitand` at `0xff`. The query puts such a pair with the parameter bound, each of the stand-in's parameters given by what the form writes in its place, and writes the binding beneath the put in the log, `binds <parameter>=<literal>`.

- **The syntaxes.** `syntactic_coherence`, a meta: the two forms are two operations, their first words apart up to the first `.`, one meaning written twice. `thread_idx_x` is `S2R` and `word_copy` is `MOV`.

- **The pragmatics.** `pragmatic_coherence`, a meta: both forms read what the launch gives, a thread's or a block's number from a special register or an extent from the constant bank below the parameters, and each means what the launch makes it. Our launches have one axis and no question reads `y` or `z`: no form across axes stands at a link of its own that carries the cases, and the lower witness puts each pair at a third form's link instead, and in a launch of one axis `y` and `z` read `0` and `1`. Their meaning is the launch's, and `vector_coherence` holds them once launches of more axes are put.

- **The data's quality.** `data_quality_coherence`, a meta: the pair was answered and the cases could not make its product whole, a value the product needs that no case reaches as it was loaded. `word_mul_add` against `word_mul` reads its addend off the third operand of a case, which holds two values.

- **The noise.** `noise_coherence`, a meta: a row of the product came back both alike and apart, decided by something outside the values the form reads. `thread_idx_x` against `word_copy` agrees where the word read is the lane's own index, and which case a lane holds is the carrier's. Signal is the small part and noise the large: the decoder writes, for every set, its pairs, those a whole product names, and the rows of their products that came back one way against those that came back both ways.

- **The qualifiers.** `qualifier_coherence`, a category of C*: a statement x that qualifies the result of a statement y, the condition under which y's result stands. Each is asked as y with x against y alone. On every case where x holds the two answer alike, and on a case where x fails y's result does not stand: a pair of them closes only at a case where x fails. `LDG.E.CONSTANT` against `LDG.E`, `ld.global.nc` against `ld.global` in PTX, qualifies a load by nothing writing the memory it reads while the kernel runs, and the C of both is one text. Every carrier holds that, the two answer alike and the pair stays open, and it closes only on a carrier that breaks it, which the C leaves undefined. Whether a load may be const is read off its chain, a store to the memory it reads, and once the chains are ordered each load's slice settles it. The put stops at its first link alike and the question stays open: where the rest of a chain fails, the pair is asked again for a lower identity to witness it. A flag glued to a test qualifies it the same way, `test_word_nonzero_and` against `test_word_nonzero` closing at a case where the flag fails, and so do `if` and `unless` on a doing word and `atomic`, no other thread coming between. The statement x is a condition, a flag: an operand one form names and the other does not that carries a value, the value `word_copy` copies or the addend of `word_mul_add`, is what the result is made of, and qualifies nothing.

- **The modifiers.** `modifier_coherence`, a category of C*: a word that changes what a form reads, writes or does, asked as the form with it against the form without it. Where the two differ both results stand, and a pair of them closes at its first case apart: `global_load_signed_byte` against `global_load_byte` closes at `0080:00a0`, and `signed_byte` is the thing it reads. A modifier qualifies nothing. Each pair of `Lstar.klq` whose two forms are one text

but for their modifiers is an entry of `qualifier_coherence` or of `modifier_coherence`, and the case it closes at sorts it: a case where x fails is a qualifier's, and a case where both results stand is a modifier's. A pair open on every carrier is a qualifier's.

- **The vectors.** `vector_coherence`, a category of C*: a word whose answer changes with where a lane sits and the shape of its launch, and never with the case it is given: `thread`, `block`, `grid`, `idx`, `dim` and `warp`, each taken at `x`, `y` or `z`. It is asked over launch shapes and not over K. The texts read a pair into it where either form reads a thread's or a block's number or an extent, and into `pragmatic_coherence` as well where both do. Put by the lower witness at a link of `word_add` in launches of blocks of 256 threads, `block_idx_z` and `block_idx_y` answer alike on every case, both 0 in a launch of one axis, and stay open; every other pair across axes closes at its first case apart: `thread_idx_y` and `thread_idx_z` against `thread_idx_x` at the second lane, `block_idx_y` and `block_idx_z` against `block_idx_x` in the second block, `block_dim_y` and `block_dim_z` against `block_dim_x`, 1 against 256, and `grid_dim_y` and `grid_dim_z` against `grid_dim_x`, 1 against 3.

- **The ranges.** `range_coherence`, a category of C*: a part or a place, a description of range, which piece of one whole a form gives: `low`, `high`, `first`, `middle`, `last`, `left` and `right`. The parts of one whole put together give the whole, an entry of I (Collapse 3): `word_mul_low` with `word_mul_high` is `wide_mul`, and `word_add_first`, `word_add_middle` and `word_add_last` are a chain of one add. The texts read a pair into it where either form names a parameter at its `.hi`, working a wide value in its pieces, the low word first and the high after it.

- **The comparisons.** `comparison_coherence`, a category of C*: a relation between operands that gives a flag, `lt`, `le`, `gt`, `ge`, `eq`, `ne`, and against 0 `zero`, `nonzero` and `negative`. Each is tied to the others by entries of I: lt(a, b) = gt(b, a), ne = ¬eq, and `zero` is `eq` against 0. The pairs `test_word_ne` and `test_word_nonzero`, `test_word_eq` and `test_word_zero`, `test_signed_word_lt` and `test_signed_word_negative`, `test_wide_eq` and `test_wide_zero`, and `test_wide_ne` and `test_word_nonzero` are each a comparison against an operand beside the same comparison against 0. The first four are `structural_coherence`, put with the operand bound to `RZ`, and each closes at a case whose operand is not 0: `eq` against `zero` and `ne` against `nonzero` of a word are one concept. The last is a wide against a word, two lines against one, and no token reading joins them. It holds two sets, and the statement is the line between them. The texts read a pair into `comparison_coherence` where both forms set a flag by a `SETP`, into `equality_coherence` beneath it where every condition the two test is `EQ` or `NE`, and into `order_coherence` where one is `LT`, `LE`, `GT` or `GE`.

- **The equalities.** `equality_coherence`, a part of `comparison_coherence`: a distance from n of 0 or not 0, n any value, `eq` and `ne` at the other operand and `zero` and `nonzero` at 0. It needs no statement of reference: two words are equal under every reading of their bits, and gnascor's tests have no signed row for any of them. Equality is purely relational, and maps directly to the relational assembly: every verdict of the protocol is an equality over cases, alike or apart, and Collapse 1's integer case is f(k) = h(k).

- **The orders.** `order_coherence`, a part of `comparison_coherence`: `lt`, `le`, `gt` and `ge` at the other operand and `negative` at 0. Each needs a statement of reference for one thing to be

less than another, a sign and a width, and no answer has given one. Its entries are `unknown_coherence` until the part reads the statement out. The statement is the delta between the two parts: `test_signed_word_lt` against `test_word_lt` is apart in the statement alone, `ISETP.LT` against `ISETP.LT.U32`, and the decoder reads the pairs apart in a statement, or across the line from an equality to an order, into `modifier_coherence`, where the statement is what moved. The floating bracket of Collapse 1 is order as well, the end the part takes its rounding.

- **The operations.** `operation_coherence`, a category of C*: a form that computes a value from its operands, a shift, an add, a multiply, a bitwise or a floating operation. An operation is one whatever its scale: a shift of a bit, of a word and of many words is a shift, and `signed_word_shr`, `word_shr`, `signed_wide_shr` and `wide_shr` are all of it. The texts read a pair into it where either form's opcode computes, `IMAD.MOV`, which moves a value, apart.

- **The commutations.** `commutative_coherence`, a category of C*: a form that gives one value with its first two sources taken either way round, an add, a multiply, a least or a most, an equality, and a bitwise table whose bits at a = 1, b = 0 and at a = 0, b = 1 agree: `LOP3.LUT` at `0xc0` is an and, and commutes. The texts read a pair into it where either form's every instruction commutes. One ask answers a form with its sources in both orders, a fold.

- **The verbs.** `verb_coherence`, a category of C*: a doing word that computes no value, tests nothing and sends control nowhere, a load, a store, a move, a copy or a read of a special register. The texts read a pair into it where either form does one: `global_load_word` against `global_load_constant_word` is two loads.

- **The negations.** `negation_coherence`, a category of C*: a word that, taken twice, gives back what it was given, x(x(y)) = y on every case: `neg`, `not`, a predicate's `!`, and `unless`, which is `if` negated. On the tree a negation is the sibling link between the two members of one node, and the decoder reads it off a pair of forms of a flag that name the same operands and answer apart whatever the link reads: every case apart, or apart only where words the link does not read decide it.

- **The timings.** `timing_coherence`, a category of C*: every time the part gives, held in one place. The stall is the soonest a result is read, walked down on the part (Step 9) with the barriers and the waits that hold a read until its result is back (P9). An operation's time is read off the curve of a task, its knees against the registers and the threads (Step 10). A vector's time is the delta of one step along it: the simplest asks put in orders drawn from a seed, one register, one lane or one operand of a case more, and the delta between the orders the cost of the step. No time is written in: each is read as a delta between orders or between steps, and a time no answer has given is `unknown_coherence`.

- **The vectors of an ask.** An ask enters the part through its vectors, a register, an address, a lane: the registers it declares, the operands of a case its code loads, and the lanes it is put over, a case a lane. Its magnitude is how much of each it takes. The query asks the simplest first, at the least magnitude, the registers its chain names, and walks up only where the part refuses the least and answers the most. Carriers of one magnitude are put in an order drawn from a seed, and the delta between the orders is the cost `timing_coherence` holds.

- **The controls.** `control_coherence`, a category of C*: what decides which form runs next, in its parts. The transfer is what is done, `exit`, `return`, a branch; the loop is the way back and the label it goes back to; the condition is what it is done under, `if` and `unless`, one node that the qualifiers' guard links to, since a guard qualifies a result and conditions a transfer; the target is where it goes, a `label`; and the dispatch is a state's next, `state_dispatch` and `state_next` where a target clocks a lane a state at a time. The texts read a pair into it where either form exits, returns, branches, calls or sets a convergence.
- **The switches.** `switch_coherence`, a category of C*: a word no case reads that places the forms a case does read, `open`, `body`, `close`, `note`, `declare` and `start`. Taken out, one changes no answer or refuses the whole program, and never moves one case. The texts read a pair into it where either form holds no instruction, nothing ending at a `;`.
- **The unknowns.** `unknown_coherence`, a member of every set of our coherence: an entry no answer has read into a set. It can be anything, and it is therefore a member of all of them until an answer reads it into one. It is the gray of P1. A pair of `Lstar.klq` the decoder reads into no set is written `unknown_coherence` beneath its verdict.
- **The cases.** K_c for each category, grown by the descent (Collapse 4).
- **The identities.** I, relations between texts. Each is witnessed by the text, and both its sides are held to each other on the part over K (P10). Each is an entry of `Lstar.klq`, `text_identity <text> = <text>`, its sides functions as the stick's manifest writes them, and its verdict beneath it: closed at the first case the host or the part answers its sides apart, open with its count of cases where both answer them alike on every case. A name of the bridge carries its stem and then what it is, an identity or an address: the identities a slice of two questions carries are `slice_identity <address>`.
- **The compositions.** Chains of forms, a category whose cases are its links' cases (Q19).
- **The record.** R, every ask with its answer, the part and the size it was read on (Q9, Q10). An ask R holds is not put again. Its rows sit in the part's `.ksc` (`ksc.oracle.tsv`), each keyed by the hashes of its code, its cases and the host's answers to them, and the registers and shape it is put with. Its word is the host's place of the first case apart, `ffffffff` where every case is alike. A timed ask is a sample of its own and put every time, and an ask the gate holds never reaches the part and has no row.

answerer	relation	read by
the host, exact	f(k) ∈ [down(k), up(k)]	P1, P2
the part alone	holds(n) ⇒ holds(m) for every m < n	the walk, Step 9
the part's clock	c(r, t) ≤ hi	the bounced sustain, Step 10
a model holding state	f(k_1 … k_n) over sequences, a stated bound on states	none yet

The last row is the stick's memory, atomic, warp and sync questions, 72 of them. A per-lane relation does not reach them: they ask a sequence of an answerer holding state, and are exact only up to a stated bound on its states (learning_a_ruleset.md).

Collapse 1. One relation for every exact answer. The host gives an exact value bracketed at the result's width w, and every ask of it is f(k) ∈ [down(k), up(k)]. An integer case is a bracket of one word, down = up, and the ask is f(k) = h(k). A floating case is a bracket of two ends, and the end the part takes is its rounding (P12). A case the C of the host leaves undefined has no bracket and gates nothing. Equality, the bracket and the refused case are one relation at three widths.

Collapse 2. One walk. A relation over an ordered set that holds below a bound and not past it is read by one walk: the last register, the launch, the fewest registers and each knee. The walk steps or bisects to the bound, with the bounced sustain where the reading carries noise and a band of 0 where it carries none.

Collapse 3. Forms read through identities. A form of 2w is a pair of forms of w joined by an entry of I: add and take by the carry and the borrow, the bitwise forms word by word, a shift right through both words. The wide row of gnascor's table is then read off the word row and I, and asked of the part as a composition, in place of a form written by hand into every ruleset. The reader's own rules are entries of I as well, and the reader consults I in place of carrying them:

select(p, 1, 0) ≠ 0 = p

- −a = 0 − a
- a shift's count is its low word below the width
- (bool)v = [v ≠ 0] over the whole of v
- p ∧ q = q ∧ p, and p ∨ q = q ∨ p
- select(p, a, b) = select(¬p, b, a)
- a test glued to a flag is the test and a join, `test_*_and` and `test_*_or`

Collapse 4. Cases and checks are one pool. With K_c open, the descent that orders the cases (P2) also chooses the case to add. The open pairs of c are its candidates (P5), and the case added is a case the most of them fail.

The scheduler. The gray entries of every set are its queue, and P11's bounds are that queue written out. Each turn:

1. R gives the gray entries.
2. The ask that settles the most of them is chosen, the descent of P2 taken across categories.
3. It is put through the asker of its answerer: the part through the device daemon's admission, one ask at a time.
4. Its answer is written to R and to every entry it settles.

A process runs again only where an entry it reads has changed. A new category, case or identity is one more pool on the queue.

While the stick's questions share no case, an ask settles its own entry and no other, and every order of the queue settles as many entries. The descent across categories chooses among asks once an ask settles more than one entry: a case added to K_c, an entry of I, or a composition.

- **Built**, the pairs' queue (`pairs pair`): the query record holds each pair's path with the hash of what its put reads, its two forms' texts, the engine's chains and the host's answers. A pair closed on its path whose reads are as they were is settled: its verdict is the record's and it is put no more. Every other pair, open, with no path, or reading what changed, is gray, and the rounds put the gray pairs alone. Within a round the ask the most pairs wait on is carried first. Each put draws the order of its links of one magnitude from the seed and its own two forms, so that no pair, put or settled, moves another's. **Measured**, seed 1: the first cycle has no hash on the record's paths and puts all 54 pairs, 123 asks the record did not hold among them; the second settles 41, puts the 13 gray pairs in 3 rounds, asks nothing new, and writes the bridge the first did.

Cost, a curve in the answerer's unit. What a writing costs is what the answerer spends to give it, read off its curve (P6, Step 10), and never its count of symbols. I gives the writings that mean the same, and the curve chooses among them: of two writings I holds alike, the part spends less on the one written. The cost falls on the step between neighboring forms. Two forms that keep their registers and their unit and differ in a modifier alone step more cheaply than two that move them, and the curve is fitted over pairs of adjacent forms. The context entries of `Lstar.klq`, the nearest link before and after that names a register a link names, are the pairs it is fitted over.

- **Built**, an arrangement's cost (`pairs cost`): each arrangement of the `.kdm` whose nodes are words `sass.krs` writes is put at a link of its operator's word in a chain of ours and held to the host, and the part's clock is read before and after it written 16 times over its own result, the clock read twice with nothing between taken off: cycles, read from `SR_CLOCKLO` by the question itself, and no launch's time. **Measured** on sm_86: all 2857 arrangements alike with the host, at 4 cycles a node. The safe word stalls each link its writer's soonest whether the next reads it or not, and every operation they use is soonest at 4: the cost counts nodes until a word stalls only where a link reads the one before it.

- **Measured**, R (`orior protocol pairs queue`, every chain the engine writes and the host computes put over all 648 cases): 441 chains, 441 alike, 0 apart, held in 282 rows of `sm_86.ksc`, a row for each distinct ask. Put again, all 441 asks are read from R and none reaches the part. The register walk's 29 asks that reach the part are read from R the same way.

- **Built**, Collapse 1: every host answer is read as a bracket, one word or `down:up`, and every case as f(k) ∈ [down, up]. The host gives every integer question one word, and over all 648 cases with no ask taken from R the 441 chains answer alike as before.

- **Built**, Collapse 2: `walk_step` and `walk_halved` in `klq_identity.cu` read the last register (R252 over 35 asks), the fewest registers and each knee. Task 1006, walked again, gives the fewest at 1 register and its knees inside the spread of its times.

- **Measured**, the queue (`pairs queue`, the stick's manifest and R): 1016 entries, 441 reading 1, 0 reading 0, 575 gray, P12's table row for row. Of the gray: 399 hold a floating value, 92 a call or element nothing types and 4 a name nothing types, 34 a statement nothing reads, 33 a quotient or remainder with no reading, 11 a built-in the host computes no answer to, and 2 a form of `sass.krs` assembled with no reading of its operands. `<folder>/queue.txt` holds each entry and what keeps it gray.

- **Measured**, I (`pairs text_identity`: each side a kernel of a stick of its own, listed by nvcc, written by the engine, computed on the host and put to the part through R): 164 identities of Collapse 3 over the stick's integer types and tests, 280 sides. All 164 are open: 160 on all 648 cases, and the 4 shift identities on 144, the cases whose count is under the width, the rest left undefined by the host's C. A shift identity is therefore witnessed only where its count's high word is 0. Two false identities put beside them, `int a + b = int a - b` and `unsigned int a ^ b = unsigned int a | b`, close at their first case apart. No side the engine writes answers apart from the host, and no identity has closed on the part.

- **Measured**, the pairs (`pairs pair`: each pair of `Lstar.klq` put to the part through R, one form standing in for the other at every link of the engine's chains that holds it and carries the cases (`carrier_flow.h`), where a later link reads what it writes, and its verdict written beneath it): of 54 pairs, 36 close at their first case apart, over 620 asks, each put at its carrier's least vector, the registers its chain names, and walked up where the part refused that and answered declaring every register, its carriers of one magnitude in an order drawn from seed 1. A link of the carrier's own, the thread's index, the case's index, the bound and the addresses, carries no case, and nothing is put there. `global_load_constant_word` in place of `global_load_word`, the two apart in a qualifier alone, answers alike at 416 links over 259776 cases, and is open: the first entry of `qualifier_coherence`. The other 17 are asked nothing: no link that carries the cases holds them where the other form can be written in. `klq_decoder` reads the log out of the query's `.qry`, where it sits beside the trace, and writes each pair's set beneath its verdict: 4 in `qualifier_coherence`, `.CONSTANT` and the three tests glued to a flag by `and`, 15 in `statement_coherence`, the width and sign of a load and the sign of a shift and of a test, and 10 in `modifier_coherence`, the comparison a test makes and `and` against `or`. With the product read at the form, each closed pair put on at the links of delta 0 until its product is whole, and each value a pair's forms write of their own put to its product, 1649 asks at seed 1 and 1673 at seed 2, the pairs carry 27 concepts and the two seeds read every set and every concept alike but one. The four signed comparisons of a word against their unsigned forms, `ge`, `gt`, `le` and `lt`, are one concept, and the four of a wide another: signed and unsigned order part exactly where the operands' signs part, whichever comparison reads them. The warp's size against a copied word and against a word of a wide are one concept, alike only where the word is 0x20, and both are `qualifier_coherence` with 0x20 their qualifier, 6 qualifiers in all and 19 unknown. The thread's index against a copied word is whole at both seeds and two concepts: the thread's index is a value of each lane, and which case a lane holds is the carrier's. With the metas read and each structure put with its parameter bound, 1702 asks at seed 1, 41 pairs close, 1 is open, 12 are asked nothing, and the pairs carry 33 concepts. 18 of the 19 in `unknown_coherence` hang from at least one meta: 13 pairs in all are `structural_coherence`, 3 `syntactic_coherence`, `thread_idx_x` against `word_copy` and the block's `y` and `z` read through a uniform register, 9 `pragmatic_coherence`, 1 `data_quality_coherence`, `word_mul_add` against `word_mul`, and 1 `noise_coherence`, the thread's index again. `test_wide_ne` against `test_word_nonzero` lights none, and stays in `unknown_coherence` alone. 4 are `negation_coherence`, the complements `zero` and `nonzero` of a word and of a wide, `eq` and `ne` of a wide, and `ge` and `lt` of a word. The other 21 are `unknown_coherence`: the 17 asked nothing, among them the indices of a thread and a block no question reads as a value, and 4 pairs of two operations: 3 a value the chain computes

against one the launch gives, the warp's size or the thread's index, and a multiply-add against its product alone.

- **Theory.** The reader consulting I in place of carrying its rules, the wide row read off the word row and I, K_c, the cost curve over pairs, and the floating brackets.

2.5.14 *P14. A system learned, and the writers that read it (P8, P9, P11, Step 9)*

The goal. The query protocol learns a system by asking it, and holds nothing about that system it did not ask. Every fact a ruleset or a machine file states of a part is the part's answer to an ask: the forms it holds, what each answers, where each form's operands lie, and when a result can be read. The protocol is the interface from the transpiler to any system. It is the same for every system, and nothing in it names a vendor.

The split.

- **The protocol asks.** Its code is `src/cu/transpiler/lstar`. The slot, the relations over the cases, the fields of each form, widening and the part's scheduler are all asks, and each answer is written to R and to the part's files.
- **A vendor's writer reads.** Its code is `src/cu/transpiler/vendor_bin_layouts/<vendor>`. It takes what the protocol learned and writes it in the vendor's container and vocabulary: the container the system accepted, the file a part's forms are kept in, and the names its compiler prints (`elf64_nvidia.tsv`, `mnemonic_nvidia.tsv`). Each writer is a thin switch or a table it picks from, and it asks nothing.
- **A fact a writer holds that the protocol can ask is a value written by hand (P11)**, and it moves to the protocol as an ask.

The part's scheduler is learned, and has nothing to do with writing. When a result can be read, what holds an instruction back until then, and for how long, are facts of the part. Its timing is part of the system being learned as much as its forms are (`timing_coherence`).

- It is asked as P9 and Step 9 ask the soonest read: an ask over forms the part has already answered, each answer kept in R and in the part's `.ksc`.
- The places in an instruction the part's scheduler reads are asked as the places of a form's operands are, and kept in the machine file beside them. So are the instruction's size and where its operation key lies. No constant names any of them.
- Every ask that learns the scheduler holds to P9's bound. It is checked against what the part has already answered before it is put, and an ask that could hold a wait the part never releases is not put.
- The gate (`cubin_safe`) holds a question to what the part has answered of its scheduler. Where the part has answered nothing, it holds it to the scheduler as the kernel the system accepted holds it.

What is to do.

1. `khw_write.c` (`src/cu/transpiler/lstar/parser`) asks the part where its scheduler and its operation key lie, and keeps each answer in the machine file. It reads none of `SASS_STALL_FIRST`, `SASS_WAIT_FIRST`, `SASS_WRITE_BARRIER_FIRST`, `SASS_READ_BARRIER_FIRST`, `SASS_STALL_LONGEST`, `SASS_WAIT_EVERY`, `SASS_BARRIER_NONE` or `SASS_OPERATION_MASK` (`sass_machine.h`), and none of the instruction's size.

2. Reading the kernel, its endings and its registers out of the container, keeping a form and writing the `.khw`, and composing a form's name from the vendor's table move from `khw_write.c` to `vendor_bin_layouts/nvidia`, each a picker over what the protocol learned.

3. `klq_identity.cu` (`src/cu/transpiler/lstar/protocol`) splits the same way: its asks stay in the protocol, and every `sass_*` and `cubin_*` call it makes moves behind a vendor's writer.

4. `cubin_safe` reads its rules on the scheduler from the part's answers in the machine file and the `.ksc`.

- **Theory.** Nothing of P14 is built.

2.6 THE PROTOCOL

the query protocol is the form every ask takes, and it is what derivation is made of. A question put to a target is an address, a qualifier and a cost bound:

[ADDRESS] -> (QUALIFIER) -> [MEASURED COST] -> BINARY RESULT (1 or 0)

The address names the target: a memory address, a URI, an API endpoint, an LLM context key, a register. The qualifier is a binary question asked at it, phrased to demand a state validation and never a data payload. One loop can therefore ask a part and a service the same question. The cost bound is the most the target may spend to answer, and no hand writes that field.

The bound is unbound first. An ask carrying no bound returns the cost instead of a bit: a measurement, never a verdict. The spread of those costs is the baseline, and the baseline is `.knf`. Every bound after that is expressed against it. No absolute figure is ever written into the machine. Binding a query to `$10ms` by hand is a scale written into the machine, which the engine is optimized against at every other point; the unbound pass derives the figure in place of choosing it.

So the protocol is two passes of one ask. The unbound pass returns a cost and tells us where to go: a cheap address is worth expanding and an expensive one is worth chaining or pruning. The bound pass returns the bit, and the bit is what a branch consumes. A missing address, a false qualifier, a timeout, an error and too many cycles all read 0 at that point, and the distinction between them survives in the baseline instead of in the bit.

Two things the unbound pass has to record. A watchdog stop is not a cost reading, it is a censored sample, and a baseline built without marking them reads low. And a baseline taken once goes stale the moment the host's load changes. The reference ask is therefore put alongside the real one and measured in the same conditions, the same way the emission order is shuffled to leave nothing to tell measurement from work by.

The baseline buys chain slicing. A chain carries a cost and one number for a whole chain names no part of it. Put the unbound ask at many cuts of the chain and the per-link costs come

out of the readings together, each one measured in the same conditions as the rest. A chain is then a profile and not a total, and the expensive link is named instead of inferred.

How the asks are ordered decides whether that works at all, and the arithmetic is measured in `utils/maint/engine/measure_check.py`. Subtracting neighboring cuts puts the noise of two measurements on a quantity the size of one link, and one link is the quantity sitting under the floor: the recovered cost carries 1 + 12543/12800 floors squared of noise against a signal of 1, which orders 56 + 9692/41993% of link pairs correctly where a coin orders 50%. Repetition, descended level by level, still orders more pairs at 6400 repeats of every cut, 97 + 143/181%, and the order of asking fixes it for far less than that.

The emission order is not a shuffle, it is a carrier. A shuffle throws away what it scrambled. This order is known to the asker and tells the part nothing: the part has no way to separate a measurement from work, and every answer is still decodable, because the order is in the record. Build it so every ask covers half the links and any two asks overlap on a quarter, and the answers come apart exactly. One ask then informs every link at once in place of one link. The squared gain over asking a link at a time is (links + 1) over four: 1 + 15541/35219 at 3 links, 4 + 31188/189499 at 15, 60.2291 + r/d at 255, and growing with the chain. Nothing beats the bound on what one answer can carry. A known order reaches that bound and asking one at a time does not, and the whole gain is that difference.

A known order also beats a drawn one, and by more the further out the reading is: 4 + 883/1053 times at the median worst-link error, 31 + 37/135 at the 95th, 145 + 5/6 at the worst of 400. 18 of 400 drawn orders do not come apart at all and cost their whole pass. An engine answering every time is held to its worst case, and a known order has the same worst case every pass by construction.

Where links contend the costs stop adding, and the solve does not say so. It returns plausible per-link numbers with the contention folded into them, and what the fit could not account for stays flat while the answers go wrong by a factor of two. What catches it is a term for contention's own shape. Contention grows as the square of how many links an ask covers and the links themselves grow as the count. An order sweeping that count therefore separates the two, and an order holding it at half gives the difference nowhere to appear. The term notices at 88% where the leftover notices none of it, and the same solve then takes the damage back out. It costs one more unknown and not one more ask.

Cross-branch comparison follows from that, and it is the reason the arrangements are all kept. Every arrangement that produces an operator is a branch, each branch slices into the same relation, and comparing them link by link gives the winning path for a given problem instead of one arrangement that suits nothing in particular. A chain reading worse as a total can hold the cheapest link for the job, and only a sliced reading can see it.

The rest of the protocol, the pair states and the mnemonics the bits resolve to, is in gnascor.md. Every step of it, what backs it and the run behind its status is in the table above, and a step changes status there and nowhere else.

The gate is the engine's own descent. Each candidate arrangement is an alignment and a relation's cases are the needle, and orior's descent places the case that prunes the most, stops where the best case prunes nothing, and leaves the survivors as its answer. The descent is planned on the host against arithmetic every system that computes agrees about, and a target is asked only the cases it placed. That is steering on what is known to be true. Survival is a conjunction,

and order cannot change a conjunction. A plan that steers badly costs speed and never a wrong survivor: gate then rank, in the engine's own words.

Cost is measured by chaining, never alone: one operation sits under the noise floor and a chain clears it by fifteen times. Chaining is also what takes the bias out. A primitive that reads worse by itself and is right for a job is then chosen for that job.

An operator is a chain of primitives and the chain can be rearranged. Every arrangement that produces the operator is kept with its cost, and a job takes the one that suits it, in place of one arrangement that suits nothing in particular. A person writing a compiler by hand affords one arrangement for each operator, because a person has to write it. Nobody writes these, and the best for an application is therefore always available.

2.7 OPEN

1. **The writings the part gives are read into nothing.** A writing of one instruction is searched for on the part for every precept and every ladder relation, and every arrangement of the `.kdm` is written from them, run and read back (the writings searched, below). `L*` is still written by hand. `sass.krs` and `ptx.krs` are read off NVIDIA's compiler: `src/cu/transpiler/vendor_bin_layouts/nvidia/scaffolding/monolith_forms.sh` asks every form the record programs' lanes decide in one program between tags, a question holding a number asked again with another, builds it once and reads each block back into a form. `ptx.krs` is read off the PTX of questions in `c.krs`'s text; `sass.krs` is read off the listing of questions in `ptx.krs`'s own text, put to `ptxas` as inline PTX, a carry in through `add.cc` and out through `addc`, a predicate in through `setp` and out through `selp`. A form every question of which reads whole and alike, assembles against the machine file and is no longer than the ruleset's is written into the ruleset; every other keeps its text and `monolith_forms.md` says why. Nothing is run on the part. Where the compiler stores a test's predicate as its negation and every instruction left is an ISETP anded with PT, each comparison is turned over, a chain through .EX whole; and a reading that drops a `.hi` the ruleset names is kept. A question holding a number outside an address is asked a third time with the number loaded, which the compiler cannot fold; that reading, assembled with the number in its place, stands where the two written-in ones do not settle. Where the written-in questions read apart only between sets of banks, a register added as IMAD.IADD and a number as IADD3, the reading that assembles for every set stands. A number in a 64-bit slot is asked whole, its high word the alternate of its low. A probe classifies a form only where the number it varies shows in the answer: a numbered probe whose number the answer does not carry is one the system folded, and it settles nothing, a value the system cannot fold deciding instead. The test names no value; which numbers a system folds is the system's own, written to the part's `.ksc` on the compile channel. A settled form must still assemble for every number it was asked, folded probes counted, or a reading drawn from register operands alone is no form of an operator a literal is asked of in a slot it cannot encode. Where a reading differs from the ruleset only in an operation's signedness and the SASS reads the same word, the system compiles the two writings to one machine code: the reading is the ruleset's form, `word_mul`'s `mul.lo.s32` the ruleset's `mul.lo.u32`, and the writing the system answers alike is written to the `.ksc` beside its folds. Of 55 forms over

440 questions, 43 of `sass.krs` and 18 of `ptx.krs` are the reading. What keeps the rest: no question reads a copy, as in a straight run the allocator names a copy's two words one register, which leaves `word_copy`, `word_set`, `sign_set` and `wide_unpack` nothing to read; `sign_select` and `wide_select` take a number in either of two slots and `SEL` takes one only in its second, and no one text assembles for every number asked; `ptx.krs` reaches `launch_load_wide` through a generic `ld`; `global_load_word_if` reads as a branch, `global_add_atomic_word` and `wide_mul` read longer than the ruleset's, `wide_mul` reads `mul.lo.s` where the ruleset holds `mul.lo.u` and is folded with nothing to prove the two one word here, and the ruleset holds no `predicate_bitand` or `predicate_bitxor`. It is the loop and it is the work. The query protocol above gives the loop its shape and nothing emits one yet. The cost bound is the open part of it: static, written into the query as `$10ms`, or dynamic, measured against a running average. The chain clock already reads a cost in the part's own time, and that reading is what a bound would be set from. That clock runs inside the SASS probe, through the toolkit, and is scaffolding (the method of the engine plan): the loop's clock is read by an ask put through `host_entry.h` like every other answer. That ask is `query_ask` (`src/cu/transpiler/lstar/protocol/query/query_ask.{h,c}`): an address and a qualifier, held, equal or advancing, returning a cost unbound and a bit bound, the cost read off a clock that is itself an address. `query_ask_check.c` holds it to memory the test owns and to the host's interrupt time at a fixed address, found advancing by the ask itself. That counter steps once a clock interrupt, half a millisecond to a millisecond, and an ask is far shorter: a cost read from it is a step or nothing. A bound set from it judges a run of asks and never one. Asks at addresses nothing has said are safe go through `query_interface_walk` (`src/cu/transpiler/lstar/protocol/query/query_interface.{h,c}`): the interface runs `query_walk` in a probe, one address after another, and an address that ends the probe is answered by the ending, the walk going on from the next address in a fresh probe. `query_interface_check.c` holds it to address 0, which ends the asker on an address fault, and to the page every Windows process shares, which answers reads and ends the asker on a put. Of that page's first sixteen words the walk finds two that advance, at 0x8 and 0x14, the interrupt time and the system time of the page's own layout. The run channel finds the part's bus the way firmware enumerates one, and never the way firmware is told to. Firmware is handed its enumeration space, the configuration address and the register offsets read from a bus standard by hand, and that is a scale written into the machine and forbidden here. The channel is given no address and no offset, and finds the enumeration space by how it answers. `host_address_ask` (`host_entry.h`) writes two words that share no bits and reads each back, sorting an address into four kinds: HOLDS gave back what it was written, plain memory; FIXED gave back one word whatever it was written, a constant such as an identifier; LIVE gave back a word the part decided, a register the part drives; NOTHING gave back all ones, nothing drove the line. A sizing register answers LIVE, since written all ones it gives back the mask of the bits it decodes, which is neither the memory that would hold all ones nor the empty line that reads all ones. An enumeration space is a region where a FIXED identifier and a LIVE sizing register repeat at one stride, and both halves are already measured with nothing told: the kind from `host_address_ask`, the stride from `period_read` (M18) or shift agreement over the sequence of kinds. The channel samples the range coarse with the classifying ask, the gate's descent

steers toward where the kinds stop being flat HOLDS or flat NOTHING and start repeating a FIXED then a LIVE, one struck record gives the stride, and the rest is read one record at a time and not one address at a time. The width of a decoded region is read last, from the mask a LIVE register gives back when it is written all ones: the part sets the width and no hand writes it. On that the part's run channel stands, the records found this way with the known order, its solve and the descent running over them, and NVRTC, nvJitLink and the CUDA runtime leave the loop once a writing is put to a record and its bit read back.

The monolith (`src/cu/transpiler/vendor_bin_layouts/nvidia/monolith.cu`) is what our compiler is held against: one program, built once by NVIDIA's compiler, holding every base precept between tags, its listing the answer key and its costs read on the part by `monolith_run`. `src/cu/transpiler/vendor_bin_layouts/nvidia/scaffolding/monolith_emit.sh` holds every block against our reader, our assembler and the word our compiler writes it with, and writes `monolith_differences.md` whole on every run. Wherever the machine file holds a form, our reader and assembler give NVIDIA's operation bits exactly. What stands between our compiler and NVIDIA's writing is a state error that compounds layer on layer, and it is fixed from the root up, each fix read off the record:

- Machine file. The fields the disassembler hides are in it: the descriptor register, the field that renames the operation, the field whose 0 drops the operand, and the operand a form holds and does not print, which keeps the bits its form was seen with. `src/cu/transpiler/vendor_bin_layouts/nvidia/scaffolding/interface_sass_unprinted.sh` asks the part what each value of such a field does and writes `interface_sass_unprinted.md` whole. A predicate the same operation leaves out of its text at PT, as a load's at bits 64 to 67, is that operand's run and its form's own bits where the text drops it; the load's holds its number inverted, and the part answers each printed predicate as written. Our reader names a form a value renames by that value: a multiplier of 0 or RZ reads IMAD.MOV, of 1 IMAD.IADD and of any other IMAD. The probe reads a relative branch target, which the disassembler prints as the address it lands on, as its distance from the instruction after it, and every branch form holds its distance at bits 34 to 81 and a predicate it prints at 87 to 90. A bit one value of which leaves an operand out of the text under another name, as BAR.SYNCALL is BAR.SYNC with its barrier left out, is the operation's; a field printed twice, as BAR.SYNC R0, R0 prints its one register, is both operands', and the assembler refuses two values for it. All 2928 forms read back under their own operation from their own encoding. With `--forms` the script asks every form holding an unprinted operand, its operands filled by their kinds, and writes `interface_sass_unprinted_forms.md`: of 437 such forms 368 are asked, and 375 of their 379 runs answer alike at every value asked. Every cubin it writes declares 255 registers a thread: a kernel refuses a register number past the count it declares, and F2FP's run at 64 to 72, refused at bits 68 to 71 under the pattern's 10, answers alike at every bit under 255. BAR.SYNC's and NANOSLEEP's runs are refused at every turned bit. A form that takes an address is asked through the word it writes, and none of the 40 atomics holding an unprinted register at 64 to 71 runs at its own bits: 22 are refused as illegal instructions, and 18 take a 32-bit or shared address that nothing

the question holds backs. All 40 were reached by turning bits. NVIDIA's compiler writes an atomic for an atomic on `.global` through a 64-bit pointer, and its 64-bit add holds the descriptor register at 64 to 69 and two bits it refuses otherwise at 70 and 71, no register. The forms with no result to read are not asked. `src/cu/transpiler/vendor_bin_layouts/nvidia/scaffolding/interface_sass_fields.sh` turns each operation bit of every form `sass.krs` uses and runs it on the part, its result moved to R8, its sources to registers holding distinct values and a predicate it sets read through `SEL`, and writes `interface_sass_fields.md` whole. A turned bit is put to the part only where its operation key holds forms in the machine file and no control transfer or wait among them; any other is marked skipped and run on nothing. Of the 75 forms, 1619 bits read inside a run the machine file records and 526 outside every run. The three branch forms name a label and are not asked.

- Scheduler bits. NVIDIA sets them an instruction at a time. A late result sits behind a write barrier and a store behind a read barrier, read from what NVIDIA's compiler writes over the tree (`monolith_scheduler.md`). The soonest a fixed result is read is the part's answer on the run channel, walked down by `pairs stall` and held in `sm_86.ksc` as `run answers <stall> stall <writer> <reader>`. Our safe word stalls each instruction the largest soonest its operation answered, the longest where it answered none (`sass_operation_schedule`). `sm_86.ksc` holds none until the stick answers whole at the longest stalls: that run is the baseline each stall is timed against, and a stall is kept at the longest whose time matches the shortest. `cubin_safe` asks no stall of an instruction a form holds: a short one gives a wrong answer and never a kernel that does not return.
- Writings searched on the part. `src/cu/transpiler/vendor_bin_layouts/nvidia/scaffolding/interface_sass_writings.sh` puts every form of the machine file that writes a register from registers, predicates and numbers alone, 745 of 2928, in place of the kernel's IADD3, each through the gate, and runs it on the part over 256 cases at once: the ladder's two-word cases, the words a width turns on against the counts a shift turns on, and words drawn as `chain_build` draws its sweep. A form is put with every assignment of the two words and RZ to its register sources that gives both, and a number whose field is eight bits wide, a truth table, with each of its 256 values. It writes `interface_sass_writings.md` whole: 5685 cubins, 5394 run and the part refuses 291. Every precept that carries a word holds under at least one form, asr, rol and ror each as a `.W` funnel given the word on both halves, and every ladder relation but same. On the ladder's cases alone a dot product of bytes holds product and every shift holds up or down: the drawn words take them out. Up and down hold only under `.W`, since the precept wraps a count of 32 and more and the forms without `.W` answer 0 there.
- Arrangements run on the part. The same script writes every arrangement of `machines/sm_86.kdm` node by node, each node the first writing the search found for its precept, runs all 3068 over the same cases and writes `interface_sass_chains.md` whole: every one answers its relation on every case, add 1202, take 1047, up 411 and down 408. Then the cases `gate_descent` places for each relation, one or two,

are put to every arrangement it descends over, 3575 of them, on the part alone, and `interface_sass_descent.md` holds the part's verdicts to the descent's: what stands on the part stands on the host for every relation, and no verdict differs.

- `sass.krs`. `word_shl` and `word_shr` carry no `.W`, and a count of 32 or more answers 0 where the precept wraps it. The machine file holds no `.W` form with a number for the count, and the code generator writes number counts of 1 to 31 alone, where both forms agree. `word_funnel_right` carries no `.W` and has no left form; there is no arithmetic shift form; NOT, NAND and NOR have no form, each one `LOP3` (0x33, 0x3f, 0x03); add and subtract write `IADD3` alone.
- Word web and alphabet web. On this part every gate is one `LOP3` node and ASR, ROL and ROR are each one `SHF` node, where the webs hold gates as separate precepts, NOT and NAND written from each other, and the rotates and the sign spread as trees the width counts. `word_shl` and `word_shr` are the precept by the word web and not by its form.
- Candidates the part's clock chooses between: ERR as a trap in place or a branch to a handler; MOV as the passage or `word_copy`; ADD and SUB as `IADD3` or `IMAD.IADD`.
- Past the monolith: NAND and NOR in one `LOP3` where NVIDIA writes two, and ROL and ROR in one `SHF` without the guard NVIDIA keeps from the source.
- The harness walks the alphabet tree one level where a precept has no word.

2. **`.kdm` costs count nodes.** `utils/maint/engine/chain_check.sh` writes 2857 arrangements over 27.6M tried, add 1140, take 904, up 409, down 404, each once, and nothing for same, places or product at three nodes. `orior protocol pairs cost` puts each one whose nodes are words `sass.krs` writes at a link of its operator's word in a chain of ours, holds it to the host, and reads the part's clock over it, every question in one round: all 2857 are costed, every one alike with the host, at 4 clocks a node. `word_nand`, `word_nor`, `word_rol` and `word_ror` are written from the forms the part answers each precept for (`interface_sass_writings.md`), and the stall walk reads each rotate's soonest in the arrangements that hold it. The safe word stalls each link the soonest its writer's result is read whether the next link reads it or not, and every operation the arrangements use is read soonest at 4: a cost counts nodes. Arrangements of one count of nodes are told apart only by a word that stalls where a link reads the one before it.

3. **The query record holds its asks and their costs, and not yet the paths over them.** `.kqr`, the query record, holds the asks put to a member and the paths read off them, in that order: every probe and what came back, with costs, refusals and censored samples each marked, then the winning path per problem over those same asks. It takes the stem the rest of the set takes. That face, `.kdm` and `.knf` under one stem are a member's coherence map and fingerprint it exactly. It carries the general and specific split `.kdm` carries: a generic block good for any member of a class, and a specific block holding the best combination available for one section of one member. Keeping the asks beside the paths leaves the fingerprint independent of `.kdm` in place of a cache of it. A refused or censored probe appears nowhere in a table of chain costs, and it separates two parts that cost the same. Every mode of `orior protocol pairs` writes `sm_86.kqr` (read and written by `kqr.c` in `src/cu/types/file_defs/`

kqr) beside R: every untimed ask and what came back, each case's word or the refusal, timed_out where the watchdog ended it and censored where the gate held it off the part to protect the whole; every timed ask a sample of its own with the nanoseconds its launches took; then each pair and its verdict. A cycle reads it first and asks the part only what no cycle asked, a timed ask excepted and an ask that timed out asked again until an answer resolves it, and an open pair is the question a further pass takes up. The curve of task 1006 writes 293 samples, 278 with their cost and 15 refused. The asks and samples stand in the order they were put, each cycle that put a new one before them with its mode, its seed where it draws one and its rounds, and a cycle the record answers whole writes no line. The winning path over chain costs and the general and specific split are not written yet.

4. **The order of asks is built on the host and nothing emits it to a target.** The order, its solve and the contention read are proved on the host (M24 in the engine table, Q5, Q7). The device half is open: a container that runs a chain's covered links and reads the part's clock around them, put through the channel in Open 1, with the censored-sample mark and the reference ask alongside. Its answer carries one bit a check, 128 an ask, and never one bit over a set (Q15).

5. **Stem membership has a written rule and nothing reads it.** Two members sharing a stem is the whole basis of a set, and pairwise agreement inside a floor cannot decide it. `src/cu/transpiler/lstar/protocol/order/stem_group.{h,c}` holds an anchored group rule: the members in an order fixed by what they are, the finest floor first, the first member with no group anchoring one, and every member with no group that agrees with that anchor joining it. Agreement is a conjunction over rows at the coarser floor, and a row one member refused and the other measured separates them. `stem_group_check.c` (run by `utils/maint/engine/chain_check.sh`) holds it: the three members `order_check.py` breaks pairwise agreement with group the same way in all six orders, and over 400 drawn sets every member agrees with its anchor, no two anchors agree, and every set groups alike under 24 shuffles. A group is a function of the whole set, and a block written for one is written again when the set changes. The open part is the general block in `.kdm` keyed to a group, which nothing writes yet.

6. **The part's ruleset is not read whole.** The code generator reads a ruleset only whole (`ruleset_core_read.h`): a bank of each of the nine its schema names (`machine_ir_types.h`), each of its fourteen fixed registers, a toolchain, a header and every one of its 165 forms. `krs_write` writes `sass.krs` and `sm_86.krs` from what the part answered alone, a form for each word answered and no bank, fixed register, toolchain or header; the code generator does not read the part's ruleset, and `pairs pair` stops before it puts a pair. Each is a fact of the part and is asked of the part (P11, P14), every round dry through the airlock first, crossed against the vendor's disassembler, and then put:

    a) The banks, by P11's Step 1 and Step 9: the width; the last register, R252, walked already; the predicates walked the same way; a wide value read back from an even pair of registers; a store and a load through the lane's address read back; and the widest number a form's number field holds. Each bank takes the cheapest place the part answers holds it (P6).

b) The fixed registers: the register that reads 0 whatever is written to it, and the lane's number, the record and the index where the kernel the system accepted holds each at its slot; the ten others the banks' asks place. **Measured** on sm_86 (`orior protocol pairs fixed`, kept in `sm_86.ksc` as `run answers 00000010 slot place`, `run answers 000000ff fixed zero`, `run answers 00000004 fixed record`, `run answers 00000002 fixed index`, `run answers 00000000 case word 0` and `run answers 00000007 case word 1`): the slot at place 16 of the kernel's 32, zero RZ, the record in R4 and the index in R2, the case's words left in R0 and R7, and no register holding the lane's number at the slot.

c) The toolchain and the header: a container the vendor's writer makes, loaded and returned from, and the least opening a kernel runs with. **Measured** on sm_86 (`orior protocol pairs frame`, kept in `sm_86.ksc` as `run answers 00000001 container returned` and `run answers 00000001 opening least`): the container returned over 648 cases, and the least opening begins at place 1 of the kernel's 32.

d) Every form the schema names: by P11's Steps 2 to 7 where the ladder holds its relation, and by a walk of its own where it holds none, as P7 asks `loop_back_if`.

e) `krs_write` writing every answer, the code generator reading the part's ruleset whole, and the bridge's pairs put: dry, crossed against the vendor's disassembler, then on the part.

2.8 THE RUN, A DATABASE (Q20)

A run is one table of blobs, and is to be a database of its own, a table for each area of it. A run's `.qry` holds every blob handed to it and a table of them, a row a blob: its name's hash, its sequence, its time, its priority and where its bytes sit (`qry.h`). Each process of the run hands into a ring of its own, which the writer moves into the run in turn; a file handed to the run is written in its place once the run ends (`qry_filed`), and while the run runs every file is read from memory (`qry_file_read`). A `.kqr` a run keeps is a view of the run's table of asks, a row each record (`kqr_tabled_by`). The run as planned:

- **A table for each area.** The teacher's asks, the carrier's launches, the gate's verdicts, the overseer's conferences and lessons, the cross's pairs, the writers' files and every trace each own a table. A row is fixed numbers: its key, the run's sequence; its who (`QryWho`), the source, the line, the program and the time; and the columns its table names, each a number. A text is held once, in the run's table of texts, and a row names it by its number, as a `.kqr` names its texts (`krep_records.h`). A table's columns and codes stand in its oracle, `<area>.oracle.tsv`, and its reader and the decoder read them from there.
- **The tables.** Past the key and the who, each row holds these columns, every one a number, a text by its number and bytes of a length a row cannot fix by the key of their blob:
  - texts: the text's bytes, each text once for each process that numbers it.
  - blobs: the blob's name, its bytes and its priority, as the `.qry`'s table holds them (`qry.h`).

  - links: the key a link is from, the key it is to and the link's code: every relation between rows that one column cannot hold, the launches a verdict read, the accounts a conference heard, the rows a file was written from.
  - asks, the teacher's: the records `.kqr` holds, a row each (`kqr.h`). An ask its identity, the hash of its code, its registers, threads, blocks and launches, its count of cases and the hash of its cases, then its answer's code, the blob of its words and the text of its refusal; a sample its identity, its answer's code, its nanoseconds and the least and the most one launch took; a cycle, a seed and a round each its number; a pair its two forms' texts, the hash of what its put reads and its verdict's text.
  - launches, the carrier's: the ask's key, the blobs of its code and its container, the outcome's code, answered, refused, timed out or faulted, its nanoseconds, the least and the most one launch took, and the text of what the carrier said.
  - verdicts, the gate's: the ask's key, the rule's number, held or put, and the text of its reason.
  - accounts, conferences and lessons, the overseer's: an account the expert's code and the text it gave; a conference the question's key, the cause's code and its ruling's code, consensus or none, the accounts it heard its links; a lesson the conference's key, the cause's code, the text of its carrier or every carrier, and the text of the stand-in as written.
  - pairs, the cross's: the texts of its two forms, the verdict's code, closed or open, its count of cases alike and the key of the case that closed it.
  - files, the writers': the text of its path and the blob of its bytes, the rows it was written from its links.

- **A trace is rows.** Every row carries its who, the source and the line of the code that made it; an area's trace is its rows in the order of their keys. A decision that makes no row of its own, an ask the gate puts past a rule, is a row of its area's table with the decision's code.

- **A buffer for each member.** Each process of a run is a member and hands its ordinary blobs into a ring of its own in the run's shared memory, at the ring's tail, with no lock another member takes (`qry_buffer.h`). The writer takes the members in turn and on a member's turn moves everything from the ring's head to its tail into the run's buffer in one go, each blob given its sequence as it goes in; a ring that holds nothing has a turn that does nothing, and the member keeps adding past what is taken. A panic and a critical blob go in at the tail as they are handed, ahead of every member. A process moves its own ring before another reads what it handed (`qry_moved`) and settles every ring once a process it started has ended (`qry_settled`).

- **One writer.** The pressure buffer drains every area's buffer into the `.qry`: panics first, then critical, then the ordinary rows in stretches, as it writes blobs. As the run ends it dumps each table's rows and its index beside the blobs, and the blobs stay a table of their own for raw bytes, a cubin or a listing.

- **Rows keyed to rows.** A row names the rows it follows from by their keys: a launch the ask it carried, a verdict the launches it read, a conference the refusals it heard, a file the rows it was written from. A question about a run is a walk over keys, and the decoder (`kdecode`) is its select: a table, a range of keys, raw or decoded.

- **Files as views.** Nothing writes a file in its place while the run runs. A file handed to the run is a row of the files table and its bytes, every program of the run reads the latest of it from the run (`qry_file_read`), and the writer writes each file in its place as the run ends. A file the run reads from the disk is taken into the run the first time it is read, and read from memory after: while the run runs, it reads from memory and not the disk. A tool outside the run that reads a file from the disk while the run runs, nvcc, nvdisasm or a script's `cmp`, is pointed at a stream the run puts out for it (`qry_streamed`), as `measuring_stick_query.c` points nvdisasm.

It is built in this order, each step held by the checks that hold the run now (`chain_check.sh`, the overseer's check, the writers' bytes):

1. A buffer for each member under the blob calls as they stand, and the time of a hand measured under contention before and after. Built and measured (`build/harness/qry/hand_time.sh`, 2000 hands of 256 bytes or 200 of 64 KiB a process): a hand of 256 bytes takes a median of 0.1 µs at 1, 4 and 16 processes against 1.2 to 1.4 µs under the one lock, and its 99th hundredth 1.4 to 2.3 µs against 5.5 to 18; a hand of 64 KiB takes a median of 31 to 41 µs against 30 to 50, its 99th hundredth 170 to 2491 µs against 56 to 341, and 16 processes hand theirs in 15.4 ms against 25.0. Every byte reaches the run as handed.

2. Files written as the run ends, and every step that reads a filed file from the disk while the run runs found and pointed at a stream. Built: the writer writes each file's latest handed bytes in its place once the `.qry` is dumped and before it says it has stopped; a file the run reads from the disk is taken into the run and read from memory after; the engine's ruleset reader reads every ruleset, `.kdm` and `.ksc` as the run holds it, a program built without the run's buffer linked to stand-ins that read the disk (`ruleset_flat.cu`); and every other read of a file the run writes, `klq_identity`'s pipe rows, the descent's cases and `sass.krs` in the scaffolding and the monolith, goes through the run. `kdm_check` holds a file the run writes to be read from the run and not the disk while the run runs, and `chain_check.sh` holds it in its place once the run has ended.

3. Rows: the head and the records of `krep_records.h` made a table of the run, the teacher's asks first, which `.kqr` holds, and `.kqr` then a view of that table. Built: a k-file a run is to write is a view of a table of the run (`krep_tabled_by`), its head a critical row whose sequence is the file's key, then a row of each text and each record it holds, and a row of every text it takes and every record put, given other words or let go after, each record's row of the who in force over it; the file written is the file its rows put together in the order the run holds them (`krep_viewed`), its rows and their numbers in `krep_records.oracle.tsv`. The teacher's R and `klq_identity`'s record are each a view of the table of asks (`kqr_tabled_by`), a text a row of the file's table as the file numbers it. `kqr_check` holds the rows to put together the record as held, every text and every record, on its own record before and after

what a run puts and on the part's three records; the largest, 15023 records, is made a view in 77 ms and put together in 142 ms, the tracked one in 6.4 ms and 2.5 ms (`build/harness/kqr/table_time.sh`).

4. The carrier's launches, the gate's verdicts, the overseer, the cross and the writers, a table each, and the table of texts every table names its texts from. A row of these is handed by one call (`qry_buffer.h`): a blob of the table's name, the who's two words, then the table's columns, each a 64-bit word, in the order the table's oracle gives (`<table>.oracle.tsv` beside `qry_buffer.h`). A text's number is the 64-bit FNV-1a hash of its bytes: each process hands a text to the table of texts the first time it numbers it, so that a text two processes number stands in the table once for each, under one number. An ask's key is the sequence of its code's blob, `question<n>.bin`, which the teacher hands and the carrier reads before it puts the question. In this order:

   a) The rows and the table of texts, and the carrier's launches and the gate's verdicts (`cubin_run.c`): a launch row for every launch the carrier makes, its ask's key, its outcome's code, answered, refused, faulted or timed out, its count of timed launches, its nanoseconds, the least and the most one launch took and the text of what the carrier said; a verdict row for every question the gate reads, put or held, its ask's key, the rule's number, held or put, the byte it held at and the text of its reason. Built: `qry_buffer.h` hands a row and numbers a text, and `qry_rows_check` holds a row to its who and its columns as handed and a text to its number and its one row a process; the carrier hands a verdict row for every question its gate reads, the airlock's pop with the part taken out and the carrier live, and a launch row for every launch, the tables' columns and codes in `verdicts.oracle.tsv`, `launches.oracle.tsv` and `texts.oracle.tsv`. The overseer's check, given the real carrier and cross, holds each verdict row of the pop to be keyed to its ask, held where a rule broke and its reason a text of the table of texts, and an ask read again with the count that clears it a second row of it. A launch row is made only where a launch reaches the part, and none has yet.

   b) The overseer's accounts, conferences and lessons (`overseer.c`). Built: every account the overseer hears is a row of the run's accounts, its expert's text, its own text and the cause it reads; every conference it holds a row of its conferences, its ruling, the accounts it heard, the lesson where one was taught and its words; and every lesson it teaches a row of its lessons, its cause, its carrier and the stand-in as written (`accounts.oracle.tsv`, `conferences.oracle.tsv`, `lessons.oracle.tsv`). The overseer knows a question by its identity and not by its code's blob, and each row names its conference by the identity's number in the table of texts. The overseer's check holds a conference's row, each of its seven accounts' rows and its lesson's row to its ruling.

   c) The cross's pairs (`klq_identity.cu`), the writers' files (`qry_filed_by`), and the links between rows. Built: every pair the pair mode keeps is a row of the run's pairs, its two forms' texts, closed or open, its count of cases alike, the case that closed it, the hash of what its put reads and its verdict's text (`pairs.oracle.tsv`); every file written is a row of the run's files, handed critical after its bytes in place of the blob that followed them, its who, its bytes' blob and its path (`files.oracle.tsv`); and a k-file written as a view of its table has its bytes' blob linked to the head row of its rows, the one link a

column of a row cannot hold so far (`links.oracle.tsv`). `who_check` holds a file's row to its who, its sequence and its path, and `kqr_check` holds the link. No pair row has been made: the pair mode has not run since.

5. `kdecode` reading the tables. Built: `kdecode --table <table> <run.qry>` reads every blob of the run in the order of their sequences (`qry_file_each`) and gives the rows of the table from `--from` through `--to`, `--count` of them at most: raw, a line of each row's key and its words in hex; decoded, a line of its key, the row's kind and each column as its oracle gives it, a key with its blob's name, a text by its number and its bytes, a code by its name in the oracle or in the oracle its gloss names, and the who named. The table of texts is given as its texts; a table with no oracle is a k-file's where the run holds its head, each row its change as `krep_records.oracle.tsv` names it and a record put named through its format's oracle, and is given raw where the run holds none. As the run ends, the writer puts into it the oracle of every table it holds, read from the tree where the run holds none of the oracle's name: a table of the run's own its `<table>.oracle.tsv` beside `qry_buffer.h`, and a k-file's table `krep_records.oracle.tsv` and its kind's oracle (`qry_write_until_stopped`). An oracle a program of the run handed as a current file stands in the run, and is written in its place. `kdecode` reads an oracle as the run holds it, the latest blob of its name, and from the tree where the run holds none. `kdecode_check.sh`, run by `chain_check.sh` on the run the checks of R, the who and the rows wrote, holds the select to what the run holds and the run decoded through its own oracles with no tree to read one from (12 checks).

2.9 THE QUERY, A CRATE

The run's lifecycle and every loop of the protocol are one Rust crate. The writer, the carrier, the cross and the teacher stay the C programs they are: the run is a database of processes, a carrier's fault takes its process alone, and one writer owns the buffer. The crate holds starting, joining, carrying into, reading, leaving, stopping and keeping a run; building each program when its sources are newer; and the order each loop runs its programs in, with the words it gives them. The crate is `orior-query` at `src/ui/query`, library `orior_query`, the same edition, license and Rust version as `orior-cli`, no dependency but the standard library. `orior-cli` depends on it and it depends on nothing of `orior-cli`: the crate gives a plain list of its commands, each its id, title, what it is for, its params and the words that run it, and the command line's catalog makes a job of each whose one step runs orior itself with those words, which the window runs and streams as it runs any job. The terminal runs the same code in its own process.

The modules. Each is one thing:

module	holds
`run`	a query run: its folder, its `.qry`, its writer's process, whether it was joined, and the files it read and may leave
`writer`	the writer's words as functions over its process: `start`, `ready`, `stop`, `runs`, `hand`, `latest`, `read`, `first`, `current`, `file`, `left`, `carry`, `tee`, `who` (`qry_writer.c`)

module	holds
`programs`	the trait the crate names the programs it needs through, by role: the C compiler, the C++ compiler, `nvcc` with `nvdisasm` and `cuobjdump` beside it, `cl` for nvcc's host compiler on Windows, `python`; and the PATH every program it starts runs with. The crate's own implementation finds each on the PATH alone
`tools`	every C program a loop runs, a row each: its name, its sources, its includes, its language and flags, how it links; built into `build/engine/<name>` where the binary is missing or a source or a header of a folder it reads is newer, objects compiled as many at once as the host has processors
`learn`	the part passes: discover, seed, turns, registers, rounds, dry, enumerate; the carrier's words, `--held`, `--lessons`, `--pop` and `--cross`
`cross`	the cross alone over the latest listed questions, carried into a run of its own, reaching no part
`pairs`	the bridge's pairs put to the part and the other modes of `klq_identity`: `stall`, `register`, `past`, `queue`, `cost`, `curve`, `text_identity` and the stick's slice, read, permute, known and broken; the decoder read over a pair run
`write`	the writers: `klq_write`, `Lstar.klq` and each language's `.klm` over rulesets; `krs_write` and `gsm_write`, each over the part's files
`stick`	the vendor's reading of the stick: the generator, nvcc where the text changed, `cuobjdump`, the read against `sass.krs`; the engine's stick held against it, each kernel's encodings read back through `nvdisasm` as many at once as processors
`kept`	the keepers: the writers' accounts of every run, crystals where the part is free, the newest runs kept, the host swept and `qry_host.h` written from the sweeps' median
`leaves`	what a run leaves held against what the tree holds, shown and asked before the writer writes it in place
`commands`	the command list, and the dispatch: words in, lines out through a sink, the code back

The programs it needs. The crate names a program by its role and never by a path, through `programs`. `orior-cli` implements the trait over its toolchains: the gcc toolchain's `gcc` and `g++`, or the llvm toolchain's `clang` and `clang++`, whichever is chosen under File, Toolchains; the cuda toolchain's `nvcc` and the programs beside it; the msvc toolchain's `cl`; the python toolchain's program; and the PATH the toolchains give every job. Where no toolchain is chosen the PATH is read as the crate's own implementation reads it. A program not found is said as the toolchains say it, the tool's name and that File, Toolchains opens its install page or takes its folder, and the loop that needs it does not start. The vendor's programs are found and never built.

The run. `Run::started(folder)`: where `QRY` names a `.qry` whose writer runs, the run is joined; else `query<n>-<datetime>.qry` is made in the folder, `n` one past the highest there, `qry_writer start` spawned in the background and `ready` waited for. Every program a loop starts is given `QRY` and `QRY_TOP`, the rest of its environment as it was. `carried(names)`: each run of the folder newest first, `carry` of the names it still lacks, then any name no run holds handed from the disk where the file is there. `read(paths)`: each handed under its path from the top of the tree. `tee(program, name)`: a program's output passed to the sink line by line and handed as `<name>.console`, the writer's `tee` word the one that hands it, the program's output piped into it. The end, in order: `left` for each file the loop named, the leaves shown and asked, `stop` and the writer waited for,

then only the newest QRY_KEPT runs of the folder kept, 64 where it is not set. A run started and not joined stops its writer however the loop ends. A run kept as its crystal is read back beside it through qry_crystal where that program is built; qry_crystal keeps its build against the engine in build/qry_crystal/engine through src/build_engine.sh, a build script, which stays.

The tools. The rows of tools: qry_writer, cubin_run, measuring_stick_query, khw_write, khw_machine_write, sass_listing_seeds, krs_write, gsm_write, klq_write, klq_decoder, klq_identity, measuring_stick_read, measuring_stick_engine and qry_sweep, each with its sources, includes and flags, and the file definitions every one of them links, qry/buffer, krep_records, ktext, kqr/record, ksc, kdm and krf, one list in the table. file_defs.sh and qry_run.sh stay for the scripts that source them and are no loop of the query, the scaffolding's and the checks'.

The loops. Each runs its programs with the words below, in the folder below, carrying, reading and leaving what its row holds.

command	builds	run in	carries, reads, leaves	runs
learn discover, seed, turns, registers, each [--base <cu>]	the writer, the carrier, the cross, khw_write, khw_machine_write; seed also sass_listing_seeds	build/engine/khw/qry	carries forms_work.txt, forms.txt, faulted.txt, overseer.txt; reads the part's .khw, .ksc and .kqr and the three tsv; hands the seeds, and the base through nvcc and cuobjdump	khw_write --<step> with the carrier's words, teed as khw_write.console
learn rounds <n>, learn dry [<n>], learn enumerate <slot>	the same	the same	the same, the machine file under build/engine/khw for dry and enumerate	khw_write with the rounds; for dry, -- dry, then the carrier --dry --list questions.txt and the gate's counts read off dry.txt
cross [--base <cu>]	the writer, the carrier, the cross	build/engine/khw/qry	the latest run holding questions.txt carried in with every code and cases it lists; forms_work.txt, forms.txt, held.txt	the carrier --dry --full --list, then the cross over it
write bridge [<ruleset>...]	klq_write	build/engine/klq/qry	reads each ruleset and Lstar.klq	klq_write, teed
pairs <mode> [<words>] --part <part> --to <language> [--dry] [--seed <n>] [--trace <folder>]	klq_identity, the carrier, the cross; klq_decoder after pair; the engine's stick for text_identity	the trace folder, else build/engine/qry	host_answers.txt, overseer.txt, faulted.txt; the part's files, the bridge, the rulesets and the stick's tsv; the .kdm for cost	every mode, traced
write krs, write gsm	krs_write or gsm_write, khw_machine_write	build/engine/krs/qry, build/engine/gsm/qry	reads the part's files and the rulesets	the writer, teed
stick, stick engine, stick read <words>	measuring_stick_read; the engine's stick	build/measuring_stick/qry, build/measuring_stick/engine/qry	the generator's .cu and tsv; nvcc only where the text changed; cuobjdump; leaves sass.krs, measuring_stick.md and engine.tsv	the read; the engine, its readings nvdisasm'd
kept, sweep	qry_sweep; qry_crystal where it can be	build/qry_kept/qry, build/qry_sweep/qry	the writers' accounts, the crystals, the kept and the sweep, each handed under its name	as the keepers do

command	builds	run in	carries, reads, leaves	runs
`query --<from> --<to> [--<part>] [--dry]`	what `write bridge` and `pairs` build	theirs	theirs	`write bridge` over the two rulesets, then `pairs pair` where a part is named

The leaves. What a run leaves is every file it may write in its place: each path it read and each the loop named as left. Before the writer stops, `qry_writer current <path>` gives each as the run holds it, held against the tree's file. A file the same is passed over. A text file apart, `.khw`, `.krs`, `.klq`, `.klm`, `.md` or `.tsv`, is shown as a line diff; a record file apart, `.kqr` or `.ksc`, as its byte counts before and after. Then the ask, a file at a time: `y`, `n`, `all` or `none` at the terminal. A loop's `--leave all|none|ask` names the answer where no one is at the terminal: `ask` where the standard input is a terminal and `all` where it is not; the window's job takes `all` unless its form says `none`. A refused file is handed back as the tree holds it with the writer's `file` word before `stop`: the one writer still writes every current file, and the tree's copy comes out unchanged. Nothing of this touches the disk from Rust.

The command line and the catalog. `orior protocol <command> <words>` for every command above. `cli.rs` dispatches every `protocol` word into `commands::run`, the lines to the terminal and the code as its exit. The catalog's `protocol` group is read off the crate's command list, a job a command, its params the command's, its one step orior itself with the words `protocol <command> ...` filled from the form. The Protocol menu of `commands.json` lists each. The window's own binary takes the command line's words as `orior-cli` does.

Errors. A program's exit code is the loop's. A program not found is said by name and nothing starts. A writer that does not come ready ends the loop before any program runs. A loop that fails stops its writer and keeps only the newest runs. A job stopped from the window is ended as every job is, the whole process tree, the writer with it, the `.qry` as the writer left it. The overseer's panics and conferences stay on the C side of the channel.

The checks. Unit tests in the crate against trees under `build/`: a run's naming and numbering and what `kept` removes; the tools table, every source glob of every row reaching at least one file of the tree, and the staleness rule; the words of each command; the leaves, a diff shown and a refused file handed back. One test builds the real writer and starts, hands, reads and stops a run. The command line's catalog test takes the `protocol` group from the crate, and the harness runs the crate's tests with the others. No shell script drives a loop: every place in a source's comment and in this table that names a loop names its command. `build_stamp.sh` stays with the engine's build and the examples that source it, `file_defs.sh` and `qry_run.sh` with the scaffolding and the checks.

- **Built.** `cd src/ui/query && cargo test`, 51 tests, and the catalog test of `orior-cli`, the harness running both. **Measured**, `learn turns` on sm_86: a round of 8184 questions, airlock 6.0 s (pop 1.0 s, cross 4.6 s), carrier 2.2 s; the carrier 7342 questions in 115 readings, 5424 distinct slot encodings.

2.10 THE FRAME ASKED (OPEN 6.3)

The two lines of a SASS ruleset's head are the part's answers. `toolchain cubin_write` says what turns the text into a program the part loads, and `header container` where the text's opening lines come from. Each is asked of the part on the run channel through the airlock and kept in the part's `.ksc` as a row, and `krs_write` writes the line only where the row stands.

The kernel asked. Both asks put the kernel the system accepted, `interface_ask`, the one every question is laid into (`elf64_nvidia.tsv`, the container rows). It loads the case's eight words, holds the ladder's relation at its slot, stores its two answer words and exits. The kernel's places come from the vendor's writer in its `kernel` mode (`khw_machine_write`), which reads them out of the container and hands them to the run as `kernel.txt`, a head of the places, the registers a thread holds and the place the least opening begins at, then one line a place; the frame asks read that blob as the teacher does and start from the whole kernel.

The container. The kernel's own code, laid into the container by the vendor's writer and put over the 648 cases with no host value held against it. An answer on every case means the part loaded the container, ran the kernel and returned, and writes `run answers 00000001 container returned`; the answers are kept as the kernel's own. Refused, nothing or held writes nothing, and the toolchain stays unanswered.

The least opening. The kernel's places before its closing are its opening. A variant drops the kernel's first k places and puts what is left, held to the whole kernel's own answers case by case; dropping fewer always holds, holds(k) ⇒ holds(j) for every j < k, and the walk reads the greatest k that still answers every case as the whole kernel does, by Step 9's halving between 0 and the kernel's count of places, a variant refused, apart or held by the gate being falsy. The answer is `run answers <k> opening least`: the place the least opening begins at, 0 where every place is needed. Each variant and its verdict is a line of the run's blob `frame.txt`.

Where. One mode of `klq_identity`, `frame` (`identity/frame.cu`), first in the pair run's opening before the register walk, each row written and its record closed before the next, and on its own as `orior protocol pairs frame`. The query crate's pairs loop runs the vendor writer's `kernel` mode into the run before `klq_identity`. Every ask goes into R as every mode's does.

What reads the rows.

- `krs_write` writes `toolchain cubin_write` where `container returned` stands and `header container` where `opening least` stands, leaves a line out where its row is missing, and says which fact the part has not answered. The reader refuses a ruleset without the two: `sass.krs` is written from a live run's rows and committed with them.
- The vendor writer's `kernel` mode hands every place and names the place `opening least` gives where the part's `.ksc` beside the machine file holds the row, 0 where it does not, and the teacher keeps the places from there: every question it puts after is as lean as the part allows, and the gate reads the places it hands. The frame asks themselves start from the whole kernel.
- The engine's SASS writing takes the opening a listing carries, the one the vendor writer hands.

The checks. `frame_check` beside the protocol's checks, built and run by `chain_check.sh`: the walk over a counted `holds`; the two rows written and read back as `krs_write` reads them; `krs_write`'s head from a `.ksc` with both rows, with one and with none; the kernel handed from k. Dry: `orior protocol pairs frame --dry` through the airlock exits 0 with its dry record, and `pairs pair --dry` still exits 0 with frame in its opening. Live on sm_86: the two rows in `sm_86.ksc`, `sass.krs` written from them by `orior protocol write krs`, both committed.

- **Built.** `frame_check`, 18 checks, under `chain_check.sh`; the frame mode first in the pair run and as `orior protocol pairs frame`. **Measured** on sm_86: the container returned over 648 cases, and the least opening begins at place 1 of the kernel's 32, the place before it a load the kernel never reads, over 6 asks.

2.11 THE FIXED REGISTERS ASKED (OPEN 6.2)

Four of the fourteen fixed registers are what the kernel the system accepted leaves at its slot, and each is the part's answer. `zero`, the register that reads 0 whatever is written to it; `lane_number`, the lane's number; `record`, the address the lane's answer is stored at; and `index`, the address its case's words are read from. The ten others, `carry`, `body`, `bodies`, `tables`, `threads`, `sign_base`, `indexed`, `one`, `ok` and `launch`, the frame never sets and no row names; each is placed by the banks' asks.

The slot. Each place after the opening's beginning is put with the encoding the kernel's unreached places hold; the place whose kernel then answers the case's second word on every case is the slot, nothing having written the register the kernel stores. `run answers <s> slot place`. The vendor's writer names the slot in the kernel listing's header from that row, as it names the opening, and `khw_write`'s first round takes it from there, putting the places only where no row stands: a place put with the unreached encoding runs the kernel without that place, and a place the addresses depend on faults the part.

Zero. The slot's own instruction is put writing the top of the register field, that place alone changed, and the kernel's own closing stores, which read that register, read it back in the record's second word. The register the kernel stores is then left as the slot found it, the case's second word, and the top reads 0 where the record is that word alone on every case, whatever the slot wrote: `run answers <n> fixed zero`. No other register is asked for it: a question changes one place no form of the machine holds, the gate's bound (`cubin_safe.h`: one instruction at most is held by no form), and the kernel's closing store, an exact form of the machine, is held by no form once changed.

What each register holds at the slot. For each register n the kernel holds, the slot's instruction is put with n as its first source and zero as its second, and the stored answer is n's value, over the 648 cases, case t on thread t. The readings are held against the host's thread number: equal to t on every case, the lane's number; stepping 16 a thread, the record's low word, and the register pair it begins the record; stepping 32, the index; equal to the case's first or second word, where the words are left; 0 or one constant on every case, nothing fixed. Rows `run answers <n> fixed lane_number`, `fixed record`, `fixed index`, `case word 0` and `case word 1`. A fixed register no register holds at the slot gets no row. Each register's reading is a line of the run's blob `fixed.txt`. The register field's bounds come from the `.ksc`'s field rows, `rd`, `ra` and `rb`, as the register walk

takes them, and the stall field beside them: the place a variant changes is put with its stall the longest and its yield bit clear, the scheduling the gate lets an instruction no form holds through with and the vendor's reader takes, as the walks' chains are put.

Where. One mode of `klq_identity`, `fixed` (`identity/fixed.cu`), after `frame` in the pair run's opening and on its own as `orior protocol pairs fixed`. It reads the kernel whole from the run's `kernel.txt` as `frame` does, and every ask goes through the airlock and into R. Dry, it says so, writes nothing and exits 0.

What reads the rows.

- `krs_write` writes each fixed line from its row, `fixed <name> = R<n>`, and `RZ` for the register that reads 0, the vendor's name for the field's top. A fixed register with no row keeps the line the ruleset gave it before the run, a hand line among them, since a line gives way only to the part's row for its register; the ten the banks' asks place stand so until then, and a user may edit them. A form is never dropped either: a word no form of the run writes keeps the form the ruleset gave it, and a form the part refuses is moved where it belongs. A register with neither is left out and named, as the head's two lines are.
- The `case word` rows are the banks' asks' to read.
- A row a run lets go without putting it back is kept in the part's `.krf` beside its `.ksc` (`krf.oracle.tsv`), a k-file of removals read and written through the run's writer like the others: `removed <type> <query> <reason> <row>`, the k-file's type the row left, the run's name as the writer names it, `replaced` or `refused`, and the row's own text whole. A `.ksc` row another row replaces under its question (`ksc_rows_replaced_traced_by`) and a fixed line a row replaces in `sass.krs` go there with `replaced`; a form the part refuses goes there with `refused` when Open 6.4 first refuses one. `krf_check`, 13 checks, holds the file under `chain_check.sh`.

The checks. `frame_check` grows a `fixed` part: a register's reading classified from a counted sequence of values, each kind once (`fixed_read.h`, shared with the mode); the rows written and read back as `krs_write` reads them; `krs_write`'s fixed lines from a `.ksc` with four rows, with one and with none, `RZ` for the top. Dry: `orior protocol pairs fixed --dry` exits 0 with its dry record. Live on sm_86: the slot, zero, record, index and case word rows in `sm_86.ksc`, `sass.krs`'s fixed lines written from them, both committed.

- **Built.** `frame_check`, 43 checks, under `chain_check.sh`; the fixed mode after the frame in the pair run and as `orior protocol pairs fixed`. **Measured** on sm_86 over 27 asks: the slot at place 16 of 32, zero RZ, the record in R4, the index in R2, the case's words in R0 and R7, and no register holding the lane's number at the slot; `sass.krs`'s zero, record and index lines are written from the rows, and the eleven others keep their hand lines until rows replace them.

2.12 THE FRAME FORMS ASKED (OPEN 6.4, THE FRAME)

The kernel the system accepted holds the frame's forms, and each is the part's answer. The first cut of Open 6.4 is the forms the kernel's own places are: `thread_idx_x` and `block_idx_`

`x`, which S2R reads; `word_set`, which MOV writes from a number; `global_load_constant_word`, the loads; `record_store_word`, the stores; `return` and `exit_if`, EXIT bare and guarded. The kernel's arithmetic places, IMAD, IMAD.WIDE and ISETP, are the ladder's and Step 6's, and answer nothing here. The ladder's relations are the host's arithmetic; a frame relation is what the host computes for a case from the launch and the case alone.

The frame round. `khw_write` puts it after the first round. Each place of the kernel from the opening on, the slot and the unreached ones apart, is put at the slot, that place alone changed, over the union's cases, every question of the round pooled and carried together through the airlock as every round is: as it stands, and with the field of the instruction word that holds each operand run of the slot's own form, as the part's `.ksc` gives the fields, set to the register the kernel stores, the field of the relation's first word set to the record's register instead, since a load or a store reads that field as an address and the record's register holds one. A `number` candidate reads only where the instruction holds the constant in its bits: a constant the place computes from the launch is no number. A place whose guard field holds a predicate is put twice, as it stands and with the guard at PT. The unreached encoding is put at the slot as one more question of the round, nothing, and a candidate whose answers are nothing's on every case is not read. The kernel loads the case's second word into the register it stores before the slot: a place that leaves that register alone answers as a load of the second word would, and answers no relation. The question fixes its launch shape, 32 threads a block, and the blocks the cases need, so that the host knows each case's lane and block. Nothing names a place beforehand: what it answers decides.

The frame relations. The candidates are what the host computes for case t: `lane`, t within its block; `block`, t's block; `load`, the case's first word or its second; `store`, a register's word standing in the record's second word, the first left as the slot found it; `exit`, the fill word the carrier leaves in a record no lane wrote, which the run channel names; and `number`, one constant on every case. The gate reads them as it reads the ladder's candidates, a conjunction over the cases, and the first survivor is the form's relation. A place answering none enters no form. The accepted kernel's places then answer S2R lane and block, MOV number twice, LDG load twice, STG store four times, and EXIT exit bare and guarded.

A frame form. A place that answered is a form of the working set under `frame.<relation>`, beside the ladder's `ladder.<relation>`, the two written alike in the answers and the working set, and goes through the turns, the register pass, the widening, the costs and the vendor's naming as the ladder's forms do. Its tuple's places are the relation's: the destination; the address; the offset or the number; the guard; the register stored.

The fields. A frame form's turning case is four cases, the threads 0, 1, 33 and 34 at 32 a block, whose lanes and blocks differ, and a turn counts where any of the four answers changes. The register runs are asked in two rounds. First every run is set to the index's register and then to the record's, the two the part's `.ksc` names in its fixed rows, both holding addresses: a run whose two answers differ, one of them the relation's, is the address, a load's a place of its tuple and a store's an operand the form holds and does not print as a place, since the record is where a store writes. No run is read as an address through a register holding none, which faults the part. Then each run left is set to 0 and to one more setting: a run inside the field the `.ksc` names `immediate` to 4, the aligned offset of the record's second word, which a load and a store answer apart and a number answers as itself, so that no load or store is put at an offset the part refuses; every other run to the register the kernel stores, and read as the ladder's are. A form that then holds no run

at a place of its relation's tuple, exit's guard apart, answered no relation it holds a field of: it is void, kept in the working set so that it is not put again, asked by no pass and laid out nowhere. A run that answers two ways and takes no place is an operand the form holds and does not print, keeping the bits it was seen with: S2R's special register. The guard is found on the variant put with PT: its turns bring the lane back, and the run that does is the guard, a predicate operand.

The naming. The vendor's writer names a frame form as it names the ladder's, from its table and names only: each operation row's reads gains the frame relations it answers for, S2R `frame.lane frame.block`, MOV `frame.number`, LD `frame.load`, ST `frame.store`, EXIT `frame.exit`; and the kind row reads the relations that take a kind, so that a relation no kind row reads takes no prefix. A predicate run prints as the vendor writes a guard, before the operation. The qualifiers a load or a store carries stand as open as they do for the ladder's forms.

`krs_write`. A table beside the word web, `frame_forms.tsv`, names only, maps each frame relation to its schema form and the order of its places: lane to `thread_idx_x to`, block to `block_idx_x to`, number to `word_set to value`, load to `global_load_constant_word to address offset`, store to `record_store_word offset from`, exit to `return` where the guard is PT and to `exit_if where` where it is a predicate. The forms land in `sm_86.krs` beside `word_add`, and a form the run hands none of keeps its line.

The checks. A C header of the frame relations' host answers, `frame_relation.h`, shared by the teacher and a check under `chain_check.sh`, holds each relation on cases of a known shape; `krs_write`'s mapping is held on a forms blob written by hand. Dry, `orior protocol learn dry 1 --leave none` puts the frame round through the airlock with nothing launched. Live on sm_86: the frame forms in `sm_86.khw` and `sm_86.krs`, both committed. The cross reads every frame question before the part does, as it reads every round.

- **Theory.** Nothing of it is built.

3

ANGLUIN'S L* AND THE LSTAR PROTOCOL

Purpose: The learner the protocol is named for, set beside the protocol term by term: what Angluin [1] proves of L*, what each of its parts is in the protocol, and where the protocol asks past it.

Scope: L* of Angluin [1], sections 1 to 4, against query_protocol_table.md. Statuses follow the engine workbook's README.md.

3.1 L*

L* learns an unknown regular set U over a finite alphabet A from a *minimally adequate Teacher*, one that answers two questions correctly (Angluin [1], section 1.1). A *membership query* puts a string and is answered yes or no. A *conjecture* puts a description of a regular set and is answered yes where it is U, and otherwise with a *counterexample*, a string in the symmetric difference of the two.

The learner keeps what it has asked in an *observation table* (S, E, T): S a prefix-closed set of strings, E a suffix-closed set, and T the answers on (S ∪ S·A)·E. A row is the answers one string gives across E. The table is *closed* where every row of S·A is a row of S, and *consistent* where two strings of S with one row give one row after every letter. A closed, consistent table gives an acceptor M(S, E, T) whose states are the rows of S, and no acceptor consistent with T and inequivalent to M has as few states (Theorem 1).

L* asks membership queries until the table is closed and consistent, conjectures M, and takes a counterexample with every prefix of it into S. Each time the table is not closed or not consistent, the count of distinct rows grows by one, and it never passes n, the states of the minimum acceptor for U. Each counterexample adds at least one state. L* ends after at most n conjectures with the minimum acceptor for U, in time polynomial in n and m, the length of the longest counterexample (Theorem 6); the table holds at most $(k + 1)(n + m(n - 1))n = O(mn^2)$ entries for an alphabet of k letters.

With no Teacher able to test a conjecture, a random sampling oracle EX() stands in: L*_a takes a sample apart from the conjecture as its counterexample, and ends with an ε-approximation of U with probability at least $1 - \delta$ after $O(n + (1/\varepsilon)(n \log(1/\delta) + n^2))$ calls (section 4, Theorem 7).

3.2 THE TERMS, SIDE BY SIDE

L*	the Lstar protocol	where	status
U, the unknown regular set	the rules of one member: what a part answers, or a language's ruleset	P11, P12	**built** for sm_86
A, the alphabet	the forms a member's machine file holds, and the cases K a form is put over	P11, Step 8	**built**
the membership query	the ask, `[address] -> (qualifier) -> [cost] -> bit`; over the bridge a relation of two forms on K	Q1, P1	**built**
yes or no	alike, apart, or gray; refused, censored and timed out each kept as itself	P1, Q9	**built**
the minimally adequate Teacher	the answerer of a category: the part for what it answers, the host for the exact answer h(k)	P12	**built**
the conjecture and its test	not asked: no answerer gives a counterexample, and K stands in for one	Step 8	**built**
the counterexample	the first case apart: a pair closed at its link and its case	Step 7	**built**
the observation table	the query record, every ask by its identity with what came back, and `Lstar.klq`, every pair with its verdict	Q9	**built**
a row of S, a column of E	a form, and the links and cases it is put at	P11	**built**
closed and consistent	no counterpart: a closed pair in the protocol is a pair answered apart, and a table closed in L*'s sense is one with no new row	P1, P13	none
M(S, E, T), the minimum acceptor	the ruleset, one form a slot ranked by its cost, and the sets of C*	P6, P11, P12	**built**
termination, at most n conjectures	none: every set is open, and a cycle asks only what no cycle asked	P12, P13, Q9	**built**
time polynomial in n and m	each identity asked once over every cycle, and each tier of a cycle asked in one round	Q9, the rounds	**measured**
EX(), ε and δ	not sampled: K is drawn from the values a concept holds, and every case of K is put	Step 8	**built**

3.3 WHERE THE PROTOCOL ASKS PAST L*

3.3.1 *A living language has no last state*

Theorem 6 ends L* because U is fixed: its minimum acceptor has n states, every counterexample brings L* a state nearer, and n bounds the run. A language that is spoken, written and revised has no fixed U. Every rule it holds is a rule of one moment of it, and a learner that has ended holds the acceptor of a language that has since moved on, with no Teacher left to give it the counterexample.

C is such a language. One text means different things in different revisions of it (ANSI C [4], C99 [5], C11 [6], C17 [7], C23 [8]):

- `auto x = 1;` gives x automatic storage and the type int in C89, which reads a declaration with no type as int; C99 refuses a declaration with no type; C23 reads `auto` as the type of the initializer.

- `int f();` declares f with its parameters unsaid in C89 through C17, and with none in C23. A definition with its parameters' types listed after the parentheses is C from C89 through C17, and is no C in C23.
- A call to a function with no declaration declares it to return int in C89, and is refused from C99 on.
- A signed integer is two's complement, ones' complement or sign and magnitude by the implementation through C17, and two's complement in C23.
- `gets` is C89's and C99's, and no C11 holds it. A variable length array is required in C99 and an implementation's choice in C11.
- `bool`, `true` and `false` are a header's macros over `_Bool` from C99, and keywords in C23, which adds `nullptr`, `constexpr` and `typeof`.

C is held to be a fixed language, and its revisions show it is a living one: a text's meaning moves from one revision to the next. Some of it does not move, as in any living language. From C89 to C23 unsigned arithmetic wraps modulo `2^N`, `sizeof(char)` is 1, `a[i]` is `*(a + i)`, `&&` and `||` stop at the first operand that decides them, and the operators bind as they did. A rule holds still where it is the best solution for its context, as wrapping arithmetic is for a machine's words, or where it is everywhere already, as two's complement was before C23 wrote it down: C23 made law of what every implementation did. These are a language's anchors. The protocol's ladder holds the same kind of anchor, relations every system that computes agrees about, 1, $1 \to 2$ among them (`ladder.h`, the coherence anchors), and a relation witnessed on the entries of every member asked is an anchor of all of them.

A learner that ended on C89 holds an acceptor that takes K&R definitions and implicit int, and refuses `bool`. Its Teacher answered for C89, and no counterexample from C23 ever reached it. In the protocol each revision is a member with entries of its own. A relation witnessed on C89's entries holds of them on every set holding them (P10, P12), and C23's entries are entries apart from them. Where one text is two forms across two members, the map between their rulesets says so: a name one text in one set and two in the other breaks, and two in one and one in the other collapses (`ruleset_core_relation.h`, the map). The record keeps every cycle's asks, an open pair is asked again in a later cycle (Q9), and a member added is asked as the others were (P13). Nothing is learned to an end, and nothing learned is undone by what comes after.

3.3.2 *The Teacher is not minimally adequate*

A minimally adequate Teacher answers every membership query, and correctly; T* of section 1.2 answers by tracing the acceptor it holds. The part holds no description the protocol can read, answers membership alone, and answers in more than two ways:

- It refuses a question, `illegal`, where its rules do not take it. The refusal is its answer, kept as the part gave it, and the question is censored in every cycle after (Q9).
- A question can harm it. The gate holds off the part what would hurt the whole, and the censor withholds a question the part has already called illegal (P9).

- It can fail to answer at all, and an ask the watchdog ended is asked again until an answer resolves it (Q9).
- The host answers only where it computes: a case its C leaves undefined has no bracket and gates nothing, and a category the host does not compute has no answerer (P12, Collapse 1).

3.3.3 *The sets in place of the conjecture*

A conjecture of L* is one acceptor, and its states are the classes of one relation, two strings alike on every column of E. L* takes two strings alike on E for one state, and only a counterexample later splits them. The protocol takes no such step. Two forms alike over K are gray and never one (P1: no answer of the part confirms two names one operation), and only a case apart is witnessed.

In place of the one relation, the protocol keeps a set of relations, C*, open at every node: equality, order, commutation, negation, the witnessed sets and the rest of the tree of P13, each holding its own `unknown_coherence` for what is not decided. A pair the bridge cannot answer is kept in one of them, where L* has no place for a question its table has not settled. The sets range over the forms of many members at once, where one table ranges over the strings of one language.

L* gains a new column only from the Teacher, by a counterexample or by an inconsistency the Teacher's answers made. The protocol finds its own distinguishing contexts: a pair is put at the links of every carrier that holds its form, and a pair no carrier separates is put at the links of a third form whose parameters name both (the lower witness, `witness_coherence`).

3.3.4 *What is bounded when nothing ends*

The bound of Theorem 6 is on one run over one fixed U. The protocol has no last run and gives no bound on the whole. Each cycle and each question has its bound:

- Each question is asked of the part once over every cycle, by its identity: its code, registers, shape, launches and cases. A cycle the record answers whole asks nothing and starts no process (Q9).
- Each question that waits on no answer but the tier's before it is written before any is put, and a tier is one round (the rounds). **Measured** on sm_86: the stall walk is two rounds, 5058 questions in the second, in 43 s with the record answering what it holds; the cost of every arrangement of the `.kdm` is one round of 6140.
- One ask answers a form over every case of K at once, 648 on sm_86, where L* puts one string a query (P5).

L* is the stationary case of the protocol, and Theorem 6 its limiting case. A learner over a fixed target that changes its answer finitely often and is right ever after identifies the target in the limit (Gold [2]), and need never know it has: it announces no end. L* adds a Teacher that tests conjectures, and with it identification in finite time, in time polynomial in n and m. A target that keeps changing is a drifting concept, and a learner of one tracks it (Helmbold and Long [3]); no bound on the whole run is owed, since the run has no whole. The protocol tracks its members. As

a member's drift goes to zero, U fixed as one revision of C or one part is, the protocol's cycles ask fewer new questions each time, and a cycle the record answers whole asks none. **Measured** on sm_86: the pair mode's second cycle answers all 384 asks of the first from the record and starts no process, and `queue` put again reads all 441 chains from R (Q9, P13).

Within a cycle a question has a work, the asks it takes in all, and a depth, the longest chain of asks each of which waits on the answer before it. No order of the asks puts a question in fewer rounds than its depth, and the protocol puts it in that many: every ask that waits on no answer but the tier's before it goes in one round.

- **Work.** The collapses bring the work down. A relation that holds on one side of a bound and not past it, the last register, the fewest registers or a knee, is one walk (Collapse 2), and halving closes it in $\lceil \log_2 N \rceil$ asks once one end that holds and one that does not are known (`walk_halved`, Step 9); `walk_step` finds those ends first, one ask a step. Every exact answer is one relation, a bracket (Collapse 1). A pair takes one ask a link, since one ask answers every case of K.

- **Depth.** The algebra as it stands sets the depth, and much of it is serial. Each halving waits on the answer before it, and a walk's depth is its work. A pair walks its links in order, the steer and its first case apart deciding the next link; the lower witness is put only once a pair has asked nothing; the band of a curve is asked again until it holds. Across pairs, and across the stalls of the soonest read, nothing waits: those go in one round each.

- **The descent** of P2 halves the candidates a case only where the case splits them evenly, and no bound on it is claimed.

3.4 STATUS

Every row of the side-by-side table marked **built** is held by code named in the row's section of the query protocol table, and its runs are there. The revisions of C as members of their own are **theory**: the tree holds one C ruleset, `c.krs`, and no member for a revision.

BIBLIOGRAPHY

[1] D. Angluin. *Learning Regular Sets from Queries and Counterexamples*. Information and Computation 75(2) 87-106. 1987.

[2] E. M. Gold. *Language Identification in the Limit*. Information and Control 10(5) 447-474. 1967.

[3] D. P. Helmbold, P. M. Long. *Tracking Drifting Concepts by Minimizing Disagreements*. Machine Learning 14(1) 27-45. 1994.

[4] American National Standards Institute. *Programming Language C*. ANSI X3.159-1989. 1989.

[5] ISO/IEC. *Programming languages, C*. ISO/IEC 9899:1999. 1999.

[6] ISO/IEC. *Programming languages, C*. ISO/IEC 9899:2011. 2011.

[7] ISO/IEC. *Programming languages, C*. ISO/IEC 9899:2018. 2018.

[8] ISO/IEC. *Programming languages, C*. ISO/IEC 9899:2024. 2024.